

Verifiable Memory: Learning Unified Memory Management with Local and Global Verifiers for Large Language Model Agents

Xiaolong Sun¹ Qichao Wang² Hangyu Li³ Liang Chen^{1,*}

¹Sun Yat-Sen University, Guangzhou, China

²Nanyang Technological University, Singapore

³Tencent, Shenzhen, China

sunxlong@mail2.sysu.edu.cn qichao001@e.ntu.edu.sg

masonhyli@tencent.com chenliang6@mail.sysu.edu.cn

*Corresponding author

Abstract

Large language model (LLM) agents must retain reusable information, control a bounded active context, and recover earlier evidence during long-horizon interaction. Existing methods commonly optimize long-term memory (LTM) and short-term memory (STM) separately, while unified policies are often trained primarily with trajectory-level feedback, which provides weak credit for individual memory decisions. We present Verifiable Memory (VerMem), a framework that represents LTM, active context, and episodic history as distinct states and controls them with one memory operation policy. Seven atomic operations let the policy add, revise, or soft-delete LTM entries; retrieve LTM into the active context; filter or summarize the active context; and restore selected episodic fragments. VerMem is initialized by supervised fine-tuning and trained with a three-stage reinforcement-learning curriculum. The local verifier scores executable memory transitions, and a global verifier assesses evidence coherence and terminal-memory consistency after task completion. These scores are combined with programmatically computed task, evidence-recall, efficiency, and constraint signals through hierarchical credit assignment. The verifiers are used only during training. Across five benchmarks and two LLM backbones, VerMem achieves the best result on the vast majority of reported metrics and consistently outperforms strong memory baselines. Under controlled online-token budgets on three interactive benchmarks, it also achieves the strongest efficiency–performance frontier among the compared methods. Code is available at <https://github.com/Sun-SYSU-24/VerMem>.

1 Introduction

In long-horizon tasks involving multi-step interaction, tool use, and complex reasoning, the performance of large language model (LLM) agents depends not only on single-step generation, but also on retaining and using task-relevant

information across decisions. Prior work uses *agent memory* to describe the information that an agent can attend to and use at a given time (Yu et al., 2026). Such information includes stored past executions and state representations formed from interaction history (Xiong et al., 2026; Goodyear et al., 2025). Agent memory is commonly divided into long-term memory and short-term memory (Yu et al., 2026). Long-term memory (LTM) persistently stores user- or task-specific knowledge for reuse in later turns, stages, or tasks (Jiang et al., 2024; Zhong et al., 2024). Short-term memory (STM) contains information in the current input context and supports ongoing reasoning, action selection, and context control (Wu et al., 2025b; Gao et al., 2025). LTM preserves information across time, while STM keeps the current reasoning state usable. Their coordination is therefore central to long-horizon agent reasoning.

Existing systems often optimize only one side of this process. STM-oriented methods organize or compress the current reasoning state (Qian et al., 2026; Li et al., 2025), whereas LTM-oriented methods build external stores for persistent recall (Zhong et al., 2024; Xu et al., 2025). Stored information, however, affects a decision only after it enters the active context at the right time, and reusing task-misaligned experience can propagate errors (Xiong et al., 2026). The AgeMem policy introduced by Yu et al. (2026) places LTM and STM operations in one policy, but its STM actions focus on retrieval, filtering, and summarization. Long-horizon tasks may also require an earlier observation, tool output, or intermediate conclusion to return from episodic history after it has left the active context. This operation differs from retrieving persistent knowledge and from compressing recent context.

Learning such behavior introduces a second difficulty. Yan et al. (2026), Zhou et al. (2025), and Yu et al. (2026) optimize memory decisions with downstream outcomes or trajectory-level feedback. These signals compare complete rollouts, but do not identify which atomic transition was useful or harmful. Process supervision and transition-level

credit improve intermediate feedback (Setlur et al., 2025; Luo et al., 2025), while MiroMind Team (2026) combines local and global verification during inference. Operation-level and trajectory-level verification have not been jointly used to train a unified memory operation policy.

These limitations leave three challenges for a unified and trainable memory operation policy. **(A) Heterogeneous memory coordination.** LTM determines what should be stored, revised, or removed. STM determines what should enter, remain in, or leave the active context. The two memory types operate over different states and time scales. Coordinating them within a shared decision process while preserving their functional boundaries remains difficult. **(B) Historical-context recovery.** The current prompt is only a budgeted view of the full task history. Early observations, tool outputs, and intermediate conclusions may become relevant again after the subgoal changes. Recovering the relevant fragments without reintroducing large amounts of irrelevant history is therefore nontrivial. **(C) Multi-granularity credit assignment.** The utility of a memory operation is often delayed and depends on later operations. The correct write may become useful only after a later retrieval, while an incorrect filter may cause failure several steps later. Training must assign operation-specific credit while preserving alignment with the complete task objective.

To address these challenges, we propose *Verifiable Memory* (*VerMem*), shown in Figure 1. *VerMem* preserves LTM, active context, and episodic history as distinct states, while one memory operation policy coordinates LTM maintenance, STM control, and historical-context recovery through seven atomic tools. Training uses a supervised fine-tuning (SFT) warmup followed by a three-stage reinforcement learning curriculum for LTM, STM, and their joint use. The local verifier evaluates each realized memory transition, and the global verifier evaluates the completed trajectory. Their separately normalized advantages are combined at every memory decision. Both verifiers are removed during inference. Across five long-horizon benchmarks and two backbones, *VerMem* consistently improves task performance and achieves a stronger efficiency–performance frontier.

The main contributions of this work are summarized as follows:

- **We propose *VerMem*, a unified agent memory management framework.** *VerMem* coordinates LTM maintenance and STM control through a single memory operation policy while preserving distinct states for LTM, active context, and episodic history. Its atomic action space further supports episode-level context selection.
- **We introduce local–global verifier-guided hierarchical credit assignment.** *VerMem* constructs operation-level local advantages and trajectory-level global advantages from stateful multi-step rollouts.

The two signals are combined at each memory decision, allowing operation quality and final task utility to jointly guide policy updates.

- **We conduct systematic evaluations across diverse complex tasks.** We compare *VerMem* with strong memory baselines on five benchmarks and evaluate its efficiency–performance trade-off under controlled online-token budgets. Ablations further examine unified LTM/STM management, local and global credit signals, and the multi-component reward design.

2 Related Work

2.1 Long-Term Memory (LTM)

Long-term memory research studies how agents preserve, organize, and reuse historical information beyond the active context. LangChain Team (2025) describes LangMem as a set of modular primitives for recording, searching, and consolidating persistent knowledge. Zhong et al. (2024) introduce MemoryBank for long-term recall and user profiling through continual memory updates. The Mem0 framework of Chhikara et al. (2025) extracts, consolidates, and retrieves salient information from extended conversations, while the A-Mem framework of Xu et al. (2025) organizes memory through dynamic indexing, linking, and evolution. These systems improve persistent-state construction, but their primary concern is how information is stored and organized. Stored knowledge affects current decisions only after it enters the active context at the appropriate stage. Incorrect or task-misaligned experiences may also propagate errors when reused (Xiong et al., 2026). This motivates joint control of persistent-memory maintenance and active-context use rather than isolated optimization of LTM.

2.2 Short-Term Memory (STM)

Short-term memory research focuses on maintaining the active context required by ongoing reasoning under a bounded context budget. Qian et al. (2026) organize reasoning traces and transient tool outputs into a compact executive memory. Li et al. (2025) develop structured schemata and activate query-relevant information for long-document understanding. Wu et al. (2025a) periodically compress growing interaction histories into concise reasoning states and further adapt agents to reason over these summaries with ReSum-GRPO. These methods improve context usability, but they remain centered on the current input, document, or ongoing trajectory. In long-horizon tasks, useful evidence may remain in the episodic history after it leaves the active context. Retrieval, filtering, and summarization alone do not fully specify which earlier episode should return to the current reasoning state.

VerMem addresses this gap by selecting task-relevant fragments from episodic history and materializing them into the active context as an explicit STM decision.

2.3 Learning Memory Policies with Verification

Reinforcement learning turns memory management into a learnable decision process. Yan et al. (2026) learn structured updates, Huo et al. (2026) expose atomic memory operations, Zhou et al. (2025) jointly learn consolidation and reasoning, and Yu et al. (2026) train a unified LTM/STM policy. These methods primarily optimize memory behavior with answer-level or trajectory-level objectives. Process supervision evaluates intermediate reasoning progress (Setlur et al., 2025), while Luo et al. (2025) and Peng et al. (2026) assign credit to agent transitions or hierarchical decisions. MiroMind Team (2026) applies local and global verification during inference. VerMem instead uses operation-level and trajectory-level verification signals to train the memory operation policy.

3 Method

We propose Verifiable Memory (VerMem), a unified framework that maintains persistent LTM, a budgeted active context, and episodic history as distinct states. As shown in Figure 1, one memory operation policy coordinates LTM maintenance, STM control, and historical-context recovery. The policy is initialized with SFT and optimized through a three-stage reinforcement learning curriculum. The local verifier supplies operation-level semantic scores, while the composite global branch supplies trajectory-level credit from programmatic and global-verifier components. Both verifiers are removed during inference.

3.1 Problem Formulation

Unified memory operation policy formulation. The fixed task specification is denoted by q . At memory-decision step t , the long-term memory M_t , active context C_t , and same-task episodic history H_t define the conceptual state and its policy-visible serialization:

$$\begin{aligned} s_t &= (q, M_t, C_t, H_t), \\ x_t &= g(s_t). \end{aligned} \quad (1)$$

Here, M_t stores persistent information for reuse in later steps or stages, C_t is the budgeted context visible to the task solver, and H_t preserves the ordered record of observations, task actions, tool outputs, intermediate conclusions, and memory decisions from the same task. Separating C_t from H_t allows information to leave the active context without being removed from that record. The function g retains the task specification and current task state, then serializes budgeted views of M_t , C_t , and same-task H_t within the 8,192-token policy-state limit used by the

protocol. It does not concatenate H_t without bound. Equation (1) therefore distinguishes the conceptual state from the bounded input presented to the policy.

Let v_t denote an operation type and ξ_t its structured arguments. The complete generated memory command is $u_t = (v_t, \xi_t)$, where $v_t \in \mathcal{V} \cup \{\emptyset\}$, and $\mathcal{V}$ denotes the seven operation types formalized in Equation (6). The null command has empty arguments. The complete command and memory transition are

$$\begin{aligned} u_t &= (v_t, \xi_t), \\ u_t &\sim \pi_\theta(\cdot | x_t), \\ \tilde{s}_t &= \mathcal{T}_m(s_t, u_t). \end{aligned} \quad (2)$$

For a valid non-null command, $\mathcal{T}_m$ commits the specified memory transition. For $v_t = \emptyset$, M_t and C_t are unchanged, and the memory-state transition is an identity transition, so $\tilde{s}_t = s_t$. The identity decision is retained in the rollout record but does not create a memory-state modification. The null command remains a trainable decision in the rollout record. Rejected non-null commands leave M_t and C_t unchanged and record the failed decision and its reason in H_t . They also incur a constraint cost. Thus, Equation (2) defines $\tilde{s}_t$ on every branch.

After the memory transition, the task solver acts on the resulting active context, the environment returns an observation, and a separate interaction transition constructs the next memory-decision state:

$$\begin{aligned} b_t &\sim \pi_\phi^{\text{task}}(\cdot | q, \tilde{C}_t), \quad z_{t+1} \sim \mathcal{P}_{\text{env}}(\cdot | b_t), \\ s_{t+1} &= \mathcal{T}_e(\tilde{s}_t, b_t, z_{t+1}). \end{aligned} \quad (3)$$

The transition $\mathcal{T}_e$ appends the task action and resulting observation to H_t and constructs the next active context. The memory transition in Equation (2) and the task/environment transition in Equation (3) have distinct roles. Repeating them produces a task trajectory τ . Unified management shares the state and task objective but does not merge LTM, active context, and episodic history into one storage structure.

Progressive training and hierarchical credit. The policy follows

$$\theta_{\text{SFT}} \rightarrow \theta_A \rightarrow \theta_B \rightarrow \theta_C = \theta^*. \quad (4)$$

Phase A trains LTM maintenance, Phase B trains STM control under distractors, and Phase C trains their coordination in complete tasks. Equation (4) shows the parameter curriculum. For decision t in trajectory k , the local verifier produces a raw operation score for an executable command, while the global branch combines post-trajectory verifier scores with programmatic task, evidence-recall, and efficiency signals. Normalization converts the two branches into $A_{t,k}^{\text{local}}$ and A_k^{global} . The executor separately records hard violations as $c_{t,k}$. Their credit is

$$A_{t,k}^{\text{hier}} = \lambda_{\text{local}} A_{t,k}^{\text{local}} + \lambda_{\text{global}} A_k^{\text{global}} - \lambda_{\text{constraint}} c_{t,k}. \quad (5)$$

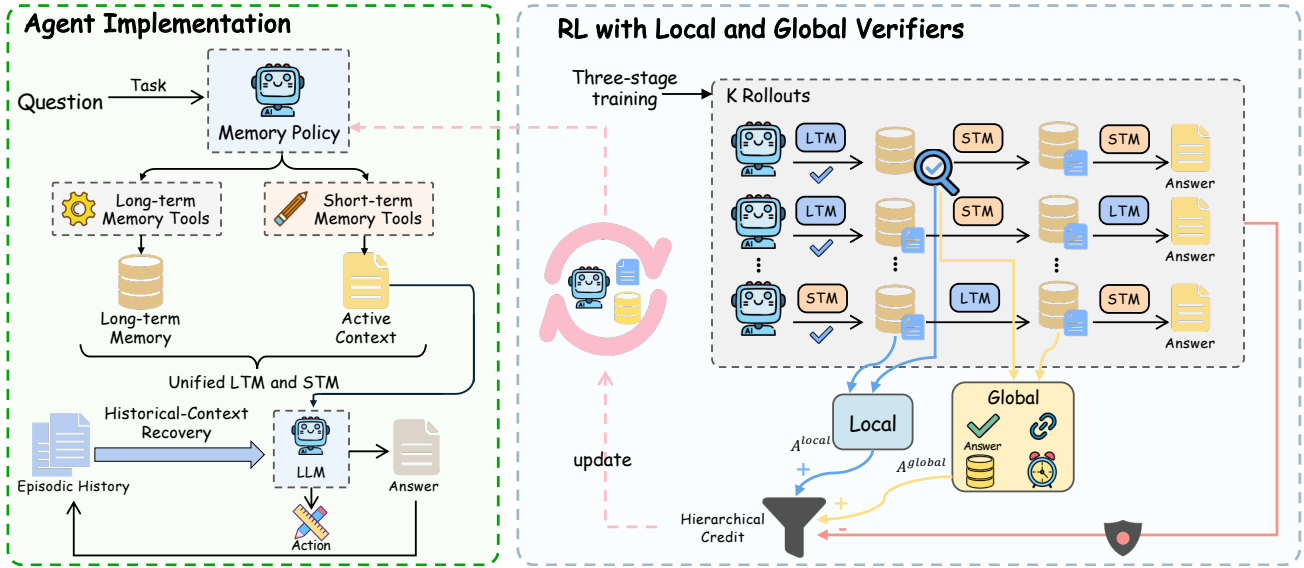


Figure 1: Overview of VerMem. Left: the memory operation policy coordinates long-term and short-term memory tools to maintain long-term memory and the active context. Historical-context recovery restores relevant episodic history for the LLM, whose actions and answers are recorded for subsequent steps. Right: a three-stage curriculum generates K stateful rollouts. The local verifier evaluates executable memory transitions, while the global verifier assesses evidence coherence and terminal-memory consistency after task completion. These scores contribute to separately normalized local and global advantages; constraint costs remain separate. The verifier branch is used only during training.

We set $\lambda_{local} = \lambda_{global} = \lambda_{constraint} = 1$. The local term distinguishes executable commands within one trajectory, while the global term preserves alignment with the completed task. Equation (5) keeps the local, global, and constraint channels distinct, including for rejected commands.

Tool	Target	Function
Add	LTM	Add to M_t
Update	LTM	Create revised versions in M_t
Delete	LTM	Soft-delete entries in M_t
Retrieve	STM	Retrieve $M_t \rightarrow C_t$
Filter	STM	Filter C_t
SelectEpisode	STM	Restore $H_t \rightarrow C_t$
Summarize	STM	Summarize C_t

Table 1: Atomic memory tools in VerMem.

3.2 Atomic Memory Tools

The operation-type sets are

$$\begin{aligned}
 \mathcal{V}_{LTM} &= \{\text{ADD}, \text{UPDATE}, \text{DELETE}\}, \\
 \mathcal{V}_{STM} &= \{\text{RETRIEVE}, \text{FILTER}, \text{SELECTEPISODE}, \text{SUMMARIZE}\}, \\
 \mathcal{V} &= \mathcal{V}_{LTM} \cup \mathcal{V}_{STM}.
 \end{aligned} \tag{6}$$

VerMem exposes these seven atomic tools, summarized in Table 1 and formalized in Equation (6), together with the null action $\emptyset$. The LTM tools add, revise, or soft-delete persistent information. The STM tools retrieve LTM content, filter or summarize the active context, and restore relevant same-task fragments from H_t through SelectEpisode. The unified action space lets the policy choose between persistent-memory maintenance and active-context control from the current state. Detailed schemas, preconditions, and transition checks are given in Supplementary Material, Section A.

3.3 Training Pipeline

Curriculum and SFT warmup. VerMem uses an SFT warmup followed by three reinforcement learning phases. The supervised set $\mathcal{D}_{SFT} = \{(x_i, v_i^*, \xi_i^*)\}_{i=1}^N$ contains validated targets for the seven tools and $\emptyset$, where ξ_i^* denotes structured arguments and $u_i^* = (v_i^*, \xi_i^*)$. The objective is

$$\mathcal{L}_{SFT} = -\mathbb{E}_{(x_i, v_i^*, \xi_i^*) \sim \mathcal{D}_{SFT}} [\log \pi_{\theta}(v_i^*, \xi_i^* | x_i)]. \tag{7}$$

Equation (7) trains both operation selection and structured argument generation. Boundary templates and candidates from a frozen DeepSeek-V3.2 (Liu et al., 2025) teacher are retained only when they satisfy the tool schema and transition constraints. The warmup teaches tool selection, argument generation, and valid structured outputs, but does not determine the delayed task utility of a memory decision. Data construction, filtering, and coverage are detailed in Supplementary Material, Section D.

Phase-wise reinforcement learning. Phase A opti-

mizes $\mathcal{V}_{\text{LTM}} \cup \{\emptyset\}$ on construction, revision, deletion, and no-change decisions. Phase B optimizes $\mathcal{V}_{\text{STM}} \cup \{\emptyset\}$ on retrieval, filtering, summarization, and historical-context recovery under distractors. Phase C enables $\mathcal{V} \cup \{\emptyset\}$ in complete multi-step tasks, where LTM and STM operations may alternate. Only memory operation policy parameters are transferred across phases; M_t , C_t , and H_t are initialized independently for every training instance.

Stateful trajectory collection. At policy update n of phase $p \in \{A, B, C\}$, the current policy samples K trajectories from the same phase-specific initial state:

$$\begin{aligned} \tau_{k,p}^{(q,n)} &\sim \mathbb{P}_{p,n} \left(\tau \mid s_{0,p}^{(q)} \right), \quad k = 1, \dots, K, \\ \mathcal{G}_{q,p}^{(n)} &= \{ \tau_{1,p}^{(q,n)}, \dots, \tau_{K,p}^{(q,n)} \}. \end{aligned} \quad (8)$$

Each rollout stores both the post-memory state $\tilde{s}_t$ and the post-interaction state s_{t+1} , so that memory-transition credit is not conflated with the subsequent task action. The candidate group in Equation (8) supports subsequent local and global credit assignment. Phase-specific states, termination rules, and token masks are provided in Supplementary Material, Sections D and F.

3.4 Local and Global Verifier-Guided Hierarchical Credit Assignment

We optimize the memory operation policy with local and global credit signals. For each task q , the K trajectories sampled from the same initial state form the candidate group $\mathcal{G}_q = \{ \tau_1^{(q)}, \dots, \tau_K^{(q)} \}$. Let $\mathcal{B}_{\text{op}}$ contain every memory decision collected in the current update, including null and rejected commands, and let $\mathcal{B}_{\text{exec}} \subseteq \mathcal{B}_{\text{op}}$ contain valid non-null commands and valid null decisions. Structural, safety, and budget violations are recorded separately as $c_{t,k}$. For $(q, k, t) \in \mathcal{B}_{\text{exec}}$, the local verifier produces $r_{t,k}^{\text{local}}$, which evaluates task relevance, evidence grounding, local progress, and information fidelity. The null decision is treated as a valid identity command and is scored with its own rubric. Rejected commands are not sent to the local verifier; their local advantage is set to zero. After task termination, the global trajectory branch produces the composite score r_k^{global} . Its verifier and programmatic components are defined below.

The composite global score is normalized within the candidate group for the same task. Local scores are normalized by operation type. The decision subset and two advantages are

$$\begin{aligned} \mathcal{B}_v &= \{ (q, k, t) \in \mathcal{B}_{\text{exec}} : v_{t,k} = v \}, \\ A_k^{\text{global}} &= \frac{r_k^{\text{global}} - \mu_q^{\text{global}}}{\sigma_q^{\text{global}} + \epsilon}, \\ A_{t,k}^{\text{local}} &= \frac{r_{t,k}^{\text{local}} - \mu_{v_{t,k}}^{\text{local}}}{\sigma_{v_{t,k}}^{\text{local}} + \epsilon}. \end{aligned} \quad (9)$$

The global statistics are computed from the K trajectories in $\mathcal{G}_q$, while local statistics are computed from operations of the same type in the current update batch.

This operation-wise normalization reduces scale differences across tool-specific verifier rubrics. When an update contains too few instances of an operation type, phase-specific running statistics are used. The corresponding thresholds and update rules are provided in Supplementary Material, Section F. The local expression in Equation (9) is defined only for decisions in $\mathcal{B}_{\text{exec}}$. For rejected commands, it is not evaluated and $A_{t,k}^{\text{local}} = 0$.

At each memory decision, Equation (5) combines the two advantages with the constraint cost. The local advantage is assigned only to the current atomic operation, while the global advantage provides task-level supervision to all operations in the trajectory. Different memory decisions within one trajectory can therefore receive different learning signals. Poorly rated operations in a successful trajectory therefore need not receive the trajectory’s full positive credit, while useful operations in an unsuccessful trajectory may retain a positive local contribution. The final sign still depends on the weighted sum of the local, global, and constraint terms. Rejected commands are not evaluated by the local semantic verifier, so $A_{t,k}^{\text{local}} = 0$. They still receive the trajectory-level A_k^{global} , while $c_{t,k}$ provides their direct operation-level penalty.

Let $y_{t,k,1:L_{t,k}}$ be the generated tokens that serialize the complete command $u_{t,k}$, including its structured arguments, and let $h_{t,k,j} = (x_{t,k}, y_{t,k,<j})$ be the token prefix. The token-level policy ratio and KL term are

$$\begin{aligned} \rho_{t,k,j}(\theta) &= \frac{\pi_{\theta}(y_{t,k,j} \mid h_{t,k,j})}{\pi_{\theta_{\text{old}}}(y_{t,k,j} \mid h_{t,k,j})}, \\ d_{t,k,j}^{\text{KL}} &= D_{\text{KL}}(\pi_{\theta}(\cdot \mid h_{t,k,j}) \parallel \pi_{\text{ref}}(\cdot \mid h_{t,k,j})). \end{aligned} \quad (10)$$

Here, θ_{old} denotes the policy frozen before the current update, and π_{ref} is the phase-specific reference policy. The coefficient ϵ_{clip} controls PPO clipping, while β_{KL} weights the KL regularization term. All command tokens share $A_{t,k}^{\text{hier}}$, but the clipped ratio and KL term in Equation (10) are evaluated token by token. The operation-normalized objective is

$$\begin{aligned} \ell_{t,k,j}(\theta) &= \min \left(\rho_{t,k,j}(\theta) A_{t,k}^{\text{hier}}, \right. \\ &\quad \left. \text{clip}(\rho_{t,k,j}(\theta), 1 - \epsilon_{\text{clip}}, 1 + \epsilon_{\text{clip}}) A_{t,k}^{\text{hier}} \right) \\ &\quad - \beta_{\text{KL}} d_{t,k,j}^{\text{KL}}, \\ \mathcal{J}_{\text{GRPO}}(\theta) &= \frac{1}{|\mathcal{B}_{\text{op}}|} \sum_{(q,k,t) \in \mathcal{B}_{\text{op}}} \frac{1}{L_{t,k}} \sum_{j=1}^{L_{t,k}} \ell_{t,k,j}(\theta). \end{aligned} \quad (11)$$

Equation (11) uses the clipped surrogate from PPO and candidate-group normalization from GRPO (Schulman et al., 2017; Shao et al., 2024). The memory operation policy loss is applied only to tokens that serialize the atomic memory command. Task-solver outputs and environment observations are excluded from this loss. Task specifications, state-serialization tokens, and padding tokens are also excluded. Operations at different time steps retain different advantages, and the factor $1/L_{t,k}$ prevents commands with longer arguments from dominating the

update. Loss masks and stabilization settings are provided in Supplementary Material, Sections B and F. The local verifier, global verifier, and reward computation are used only during training.

3.5 Reward Function Design

The training signal has three explicitly separated channels: semantic quality for executable memory commands, global utility for the completed trajectory, and hard execution constraints. Let T_k be the number of memory-decision opportunities in τ_k , including null and rejected commands, and let $\mathcal{E}_k$ index its executable commands, including valid null decisions. Null decisions count toward the decision horizon but not as memory-tool calls. Keeping the three channels separate prevents a hard rejection from also receiving an undefined or duplicated semantic penalty.

Local operation reward. The local component is

$$R^{\text{local}}(\tau_k) = \frac{1}{T_k} \sum_{t \in \mathcal{E}_k} r_{t,k}^{\text{local}}. \quad (12)$$

Equation (12) normalizes by all memory-decision opportunities while summing only scores produced for executable decisions. It is therefore zero when $\mathcal{E}_k$ is empty and is used for analysis and monitoring, not as a substitute for the operation-wise advantages. The score evaluates only the current atomic memory operation and its realized state transition. It measures task relevance, evidence grounding, local progress, and information fidelity. For LTM operations, it evaluates whether information has persistent value, whether an update agrees with new evidence, and whether a deletion is justified. For STM operations, it evaluates whether retrieval or episode-level context selection supplies missing information and whether filtering or summarization preserves useful evidence while reducing context load. Structural validity is excluded from this semantic score and is handled by the auxiliary channel.

Composite global trajectory reward. The composite global branch is computed after task termination:

$$r_k^{\text{global}} = \frac{1}{4} (r_k^{\text{task}} + r_k^{\text{evid}} + r_k^{\text{state}} + r_k^{\text{eff}}). \quad (13)$$

All four terms lie in $[0, 1]$. The task score r_k^{task} and annotated supporting-fact recall r_k^{sup} are computed programmatically. The global verifier returns only an evidence-coherence score v_k^{coh} and a terminal-memory-consistency score v_k^{state} . The evidence and state components are

$$\begin{aligned} r_k^{\text{evid}} &= \frac{1}{2} (r_k^{\text{sup}} + v_k^{\text{coh}}), \\ r_k^{\text{state}} &= v_k^{\text{state}}. \end{aligned} \quad (14)$$

Equations (13) and (14) make the boundary explicit: the global verifier supplies only v_k^{coh} and v_k^{state} , while the composite global branch also includes programmatic task correctness, supporting-fact recall, and efficiency. Terminal-memory consistency requires current active LTM and context to agree with observed evidence and treats H_t

as an ordered provenance record: superseded or failed events may remain in history if their status is explicit. Since policy training uses only HotpotQA, r_k^{task} is derived from answer correctness. Environment success, planning progress, and goal-state satisfaction are used as evaluation metrics on the downstream agent benchmarks rather than as policy-training rewards.

Let $\bar{C}_k^{\text{online}} \in [0, 1]$ denote the programmatically computed online cost from tokens, task steps, and executed non-null memory-tool calls. These components are separately min-max normalized within the K candidates for the same task and then averaged. The efficiency term is

$$r_k^{\text{eff}} = \mathbb{I}[r_k^{\text{task}} \geq \delta] (1 - \bar{C}_k^{\text{online}}). \quad (15)$$

We use $\delta = 0.5$. This gate prevents low-cost failures from receiving an efficiency reward in Equation (15).

Auxiliary constraints. The auxiliary cost is

$$P_{\text{aux}}(\tau_k) = \frac{1}{T_k} \sum_{t=1}^{T_k} c_{t,k}. \quad (16)$$

Equation (16) averages direct constraint costs over the memory-decision horizon. The step cost records discrete or hard violations, including invalid tools or arguments, failed state transitions, unsupported writes or updates, unjustified deletions, making protected evidence operationally inaccessible within the remaining decision and token budget, context overflow, exceeding the tool or interaction budget, and information leakage in no-reference settings. Routine token, step, and tool costs do not enter this channel; they are evaluated continuously by the global efficiency term. Terminal budget violations are attached to the final memory decision that precedes termination.

The scalar displayed in the All>Returns training curve is used only for monitoring:

$$m_k^{\text{all}} = \text{clip}\left(\frac{R^{\text{local}}(\tau_k) + r_k^{\text{global}} - P_{\text{aux}}(\tau_k)}{2}, 0, 1\right). \quad (17)$$

The Answer-Only curve analogously displays

$$m_k^{\text{ans}} = \text{clip}(r_k^{\text{task}} - P_{\text{aux}}(\tau_k), 0, 1). \quad (18)$$

Equations (17) and (18) are strategy-specific monitoring values. They are not policy advantages and are not directly comparable in absolute level. During policy optimization, the local and global signals are normalized separately, while $c_{t,k}$ is assigned directly to the corresponding decision. Tool-specific rubrics, normalization details, completion thresholds, and constraint triggers are provided in Supplementary Material, Section F.

4 Experiments

4.1 Experimental Setup

Datasets. We evaluate VerMem on ALFWorld (Shridhar et al., 2021), SciWorld (Wang et al., 2022), PDDL (Ma

et al., 2024), BabyAI (Chevalier-Boisvert et al., 2019), and HotpotQA (Yang et al., 2018). VerMem is fine-tuned only on the HotpotQA training split, whose supporting facts and distractors support LTM maintenance, STM control, and historical-context recovery, and is then evaluated directly on all five benchmarks. Memory states are isolated by task instance. Each episode starts with empty M_0 and H_0 , while C_0 contains only the task instruction and initial input.

Evaluation Metrics. We report Success Rate (successful episodes divided by evaluated episodes) on ALFWorld, SciWorld, and BabyAI, Progress Rate on PDDL, and the Qwen-Max LLM-as-a-Judge score J_{judge} on HotpotQA. The main table reports $100J_{\text{judge}}$. Qwen-Max is used only as a held-out evaluator; it is not used for policy training, verifier scoring, or checkpoint selection. Let S^* be a target macro-average Success Rate and B^* an online-token budget. $\text{TN}@S^*$ is the linearly interpolated online-token budget at which the mean success curve first reaches the target S^* , using the two adjacent evaluated budget points that bracket the target. $\text{SR}@B^*$ is the macro-average Success Rate at B^* . The reward ablation additionally reports Memory Quality (MQ), average online token number (TN), and average memory-tool calls (TC).

Baselines and Backbones. Base uses the common task solver without an explicit memory operation policy or external persistent-memory store. We compare it with LangMem (LangChain Team, 2025), A-Mem (Xu et al., 2025), Mem0 and its graph-based variant Mem0^g (Chhikara et al., 2025), and AgeMem (Yu et al., 2026) under Qwen2.5-7B-Instruct (Qwen et al., 2024) and Qwen3-4B-Instruct (Yang et al., 2025). VerMem-noVerify is a matched internal control that removes semantic feedback from the local and global verifiers while retaining the same action space, SFT initialization, curriculum, task-outcome reward, transition constraints, and GRPO configuration. Complete implementation and baseline settings are provided in Supplementary Material, Sections C and F.

4.2 Main Results

Overall comparison. Table 2 reports the main results under both backbones. With Qwen2.5-7B-Instruct, VerMem obtains an average score of 48.01. This is 19.96 points above Base, corresponding to a relative gain of 71.16%, and 6.05 points above the strongest external method, AgeMem. VerMem ranks first on ALFWorld, SciWorld, BabyAI, HotpotQA, and Average. PDDL is the only exception, where VerMem-noVerify is 0.35 points higher. VerMem still exceeds the strongest external PDDL baseline by 3.67 points. With Qwen3-4B-Instruct, VerMem ranks first in all six reported columns and reaches an average score of 59.85, outperforming AgeMem by 5.54 points. The consistent improvement is observed with both evaluated backbones rather than only one of them.

Performance across task types. The improvements

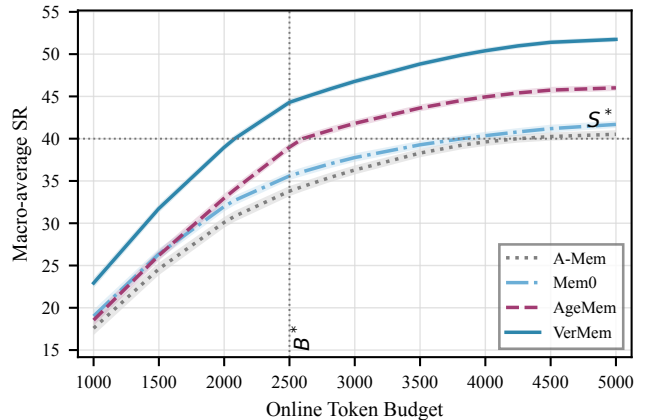


Figure 2: Efficiency–performance frontier under different online token budgets on ALFWorld, SciWorld, and BabyAI using Qwen2.5-7B-Instruct. Curves compare A-Mem, Mem0, AgeMem, and VerMem, and report macro-average Success Rate over three random seeds. Shaded regions denote the standard deviation across seeds. The horizontal and vertical dotted lines mark $S^* = 40.0$ and $B^* = 2,500$, respectively.

vary with the memory requirements of each task. With Qwen2.5-7B-Instruct, VerMem exceeds the strongest external baseline by 5.23, 7.84, 3.67, 4.13, and 8.31 points on ALFWorld, SciWorld, PDDL, BabyAI, and HotpotQA. The corresponding gains with Qwen3-4B-Instruct are 4.76, 6.89, 3.68, 4.21, and 8.12 points. The largest gains occur on HotpotQA and SciWorld. This pattern is consistent with the greater need for multi-hop evidence aggregation and long interaction-state tracking in these tasks. The smaller gain on PDDL is consistent with its more structured state representation and feedback, although the present experiments do not isolate this factor.

Cross-task transfer. VerMem is fine-tuned only on HotpotQA, yet it consistently improves over the strongest baselines on ALFWorld, SciWorld, PDDL, and BabyAI. The gains range from 3.67 to 7.84 points with Qwen2.5-7B-Instruct and from 3.68 to 6.89 points with Qwen3-4B-Instruct. These environments require environment tracking, scientific-progress maintenance, planning-state control, and instruction execution. The transfer results indicate that the learned memory operation policy captures reusable memory behavior rather than task-specific document patterns.

Efficiency–performance frontier. Figure 2 and Table 3 show that VerMem maintains the highest macro-average SR across the evaluated budgets. Its mean curve first reaches $S^* = 40.0$ at the linearly interpolated estimate of 2,080 online tokens, reducing $\text{TN}@S^*$ by 20.0% relative to AgeMem, and achieves an $\text{SR}@B^*$ of 44.30 at $B^* = 2,500$, exceeding AgeMem by 5.30 points. Both verifiers are disabled during evaluation, so these measurements do not include inference-time verification calls.

Method	ALFWorld	SciWorld	PDDL	BabyAI	HotpotQA	100 J_{judge}	Average
Qwen2.5-7B-Instruct							
Base	27.16	13.80	10.15	50.80		38.36	28.05
LangMem	38.27	28.29	15.85	51.34		37.43	34.24
A-Mem	34.68	28.06	18.39	58.82		43.95	36.78
Mem0	37.49	26.99	13.96	60.58		46.66	37.14
Mem0 ^g	35.34	30.50	14.86	58.78		42.06	36.31
AgeMem	<u>41.07</u>	<u>35.55</u>	17.31	<u>61.42</u>		<u>54.44</u>	<u>41.96</u>
VerMem-noVerify	40.18	33.94	22.41	59.21		53.08	41.76
VerMem (Ours)	46.30	43.39	<u>22.06</u>	65.55		62.75	48.01
Qwen3-4B-Instruct							
Base	38.51	47.89	30.14	55.83		47.48	43.97
LangMem	40.89	50.42	28.42	53.80		42.70	43.25
A-Mem	34.31	50.14	34.41	61.35		48.48	45.74
Mem0	41.17	51.38	31.72	60.05		39.16	44.70
Mem0 ^g	36.69	47.76	29.61	57.59		38.12	41.95
AgeMem	<u>48.97</u>	<u>59.48</u>	<u>35.07</u>	<u>72.56</u>		<u>55.49</u>	<u>54.31</u>
VerMem-noVerify	48.63	58.92	34.76	72.11		53.79	53.64
VerMem (Ours)	53.73	66.37	38.75	76.77		63.61	59.85

Table 2: Main results under two LLM backbones. We report Success Rate on ALFWorld, SciWorld, and BabyAI, Progress Rate on PDDL, and 100 J_{judge} on HotpotQA. Each entry is the mean over three independent runs with seeds 42, 43, and 44. Average is the unweighted arithmetic mean of the five displayed scores. The best result in each backbone setting is shown in bold, and the second-best result is underlined.

Method	TN@ S^* ↓	SR@ B^* ↑
A-Mem	4,250	33.80
Mem0	3,820	35.60
AgeMem	2,600	39.00
VerMem	2,080	44.30

Table 3: Efficiency–performance comparison under controlled online-token budgets using Qwen2.5-7B-Instruct, with $S^* = 40.0$ and $B^* = 2,500$. TN@ S^* is linearly interpolated from the two adjacent evaluated budget points that bracket S^* . Lower TN@ S^* and higher SR@ B^* are better.

Variant	ALFWorld SR	SciWorld SR	HotpotQA 100 J_{judge}
VerMem-noVerify	40.18	33.94	53.08
+Local	44.72	39.81	58.26
+Global	43.31	37.96	60.18
+Local+Global	46.30	43.39	62.75

Table 4: Ablation of the local and global credit branches. VerMem-noVerify uses a normalized task-outcome advantage and constraints. +Local adds the local operation advantage. +Global replaces the task-only trajectory advantage with the composite global trajectory advantage. Local+Global uses both the local and composite global advantages and denotes full VerMem. Architecture, SFT initialization, training data, constraints, and GRPO settings are otherwise shared.

4.3 Ablation Studies

Unified memory components. Figure 3 shows cumulative gains from LTM operations, the complete LTM/STM action space, stateful RL, and verifier-guided credit. LTM provides the largest initial improvement, STM control adds further gains on SciWorld and HotpotQA, and full VerMem improves over +noV by 6.12, 9.45, and 9.67 points. The progression is consistent with contributions from both the unified action space and verifier-guided optimization.

Local and global credit branches. Table 4 shows complementary effects. Local feedback yields larger standalone gains on ALFWorld and SciWorld, whereas the global branch yields a larger standalone gain on HotpotQA. This pattern is consistent with, but does not by itself prove, the intended distinction between operation-level and trajectory-level credit. Their combination performs

best on all three evaluated tasks.

Reward function. The monitoring curves in Figure 4 suggest earlier stabilization and lower late-stage variability for All>Returns. Because the two strategies use different monitoring definitions, their absolute reward levels should not be compared directly. Table 5 shows that All>Returns raises J_{judge} from 0.531 to 0.628 and MQ from 0.491 to 0.673, while reducing TN by 7.14%. TC increases from 4.47 to 5.63; together with the higher J_{judge} and MQ, this shows that the token reduction is not obtained merely by suppressing all memory-tool calls.

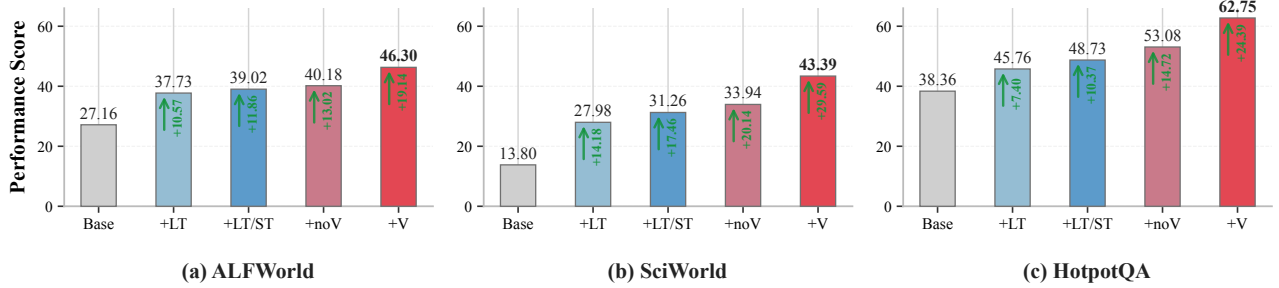


Figure 3: Cumulative component ablation with Qwen2.5-7B-Instruct. ALFWorld and SciWorld report Success Rate. HotpotQA reports $100J_{\text{judge}}$. Base has no explicit memory operation policy. +LT adds LTM operations. +LT/ST adds the full LTM/STM tool space. +noV adds stateful RL without semantic verifier feedback. +V adds the local and global credit branches.

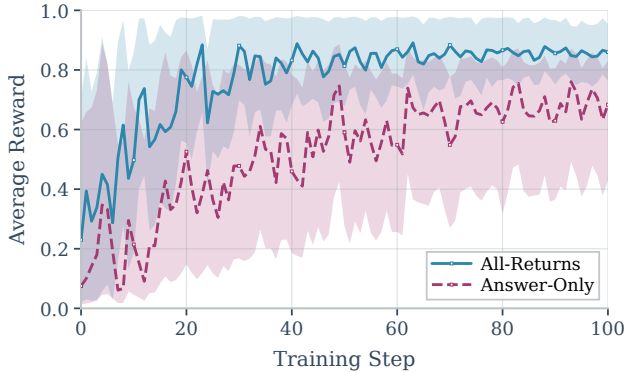


Figure 4: GRPO training-reward convergence on HotpotQA using Qwen2.5-7B-Instruct. Training Step indexes the 101 recorded training points over the 3,500-update GRPO schedule, with adjacent points separated by 35 policy updates. Curves show the unsmoothed mean over three runs, and shaded regions denote one standard deviation. Answer-Only and All>Returns use the monitoring mappings defined in the reward section; their absolute levels are not directly comparable because the mappings differ.

5 Limitations

Our evaluation resets all memory states after each episode, so it measures within-episode persistence and cross-benchmark transfer of the learned policy rather than cross-session or cross-user memory retention. The memory operation policy is trained on constructed HotpotQA states and evaluated with two Qwen backbones; the findings may not transfer to other model families or domains. HotpotQA J_{judge} and MQ use a fixed proprietary Qwen-Max snapshot, which limits fully independent replication despite disclosure of the snapshot and prompts. The frozen DeepSeek-V3.2 teacher and verifiers also introduce offline training cost. Finally, learned verifier scores may inherit evaluator errors, and the present study does not establish robustness under adversarial, privacy-sensitive, or conflicting memory content.

Strategy	$J_{\text{judge}} \uparrow$	TN $\downarrow$	MQ $\uparrow$	TC
Answer-Only	0.531	2,352	0.491	4.47
All>Returns	0.628	2,184	0.673	5.63

Table 5: Reward-function ablation on HotpotQA using Qwen2.5-7B-Instruct. J_{judge} denotes the LLM-as-a-Judge score on $[0, 1]$, TN is the average online token number, MQ is Memory Quality, and TC is the average number of memory-tool calls.

6 Conclusion

We presented Verifiable Memory (VerMem), a unified framework for learning LTM and STM management in long-horizon LLM agents. VerMem maintains persistent memory, active context, and episodic history as distinct states, while one memory operation policy coordinates seven atomic tools for persistent-memory maintenance, active-context control, and historical-context recovery. Training combines an SFT warmup with a three-stage reinforcement learning curriculum. The local verifier evaluates realized atomic memory transitions, while the global verifier evaluates completed trajectories and terminal memory states. Both verifiers are removed during inference.

Experiments across five benchmarks and two backbones show that VerMem consistently outperforms strong memory baselines and transfers from HotpotQA to interactive, planning, and instruction-following tasks. The budget-controlled evaluation further demonstrates a stronger efficiency–performance trade-off under limited online tokens. The ablation results confirm the contributions of unified LTM/STM control, stateful reinforcement learning, complementary verifier signals, and the complete reward design. These findings show that coordinated memory management and multi-granularity credit assignment provide an effective basis for reliable long-horizon agent behavior.

References

- Maxime Chevalier-Boisvert, Dzmitry Bahdanau, Salem Lahlou, Lucas Willems, Chitwan Saharia, Thien Huu Nguyen, and Yoshua Bengio. BabyAI: A platform to study the sample efficiency of grounded language learning. In *International Conference on Learning Representations*, 2019. URL <https://openreview.net/forum?id=rJeXCo0cYX>.
- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory, 2025. URL <https://arxiv.org/abs/2504.19413>.
- Pengyu Gao, Jinming Zhao, Xinyue Chen, and Yilin Long. An efficient context-dependent memory framework for LLM-centric agents. In Weizhu Chen, Yi Yang, Mohammad Kachuee, and Xue-Yong Fu, editors, *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 3: Industry Track)*, pages 1055–1069, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics. doi: 10.18653/v1/2025.naacl-industry.80. URL <https://aclanthology.org/2025.naacl-industry.80/>.
- Lyle Goodyear, Rachel Guo, and Ramesh Johari. The effect of state representation on LLM agent behavior in dynamic routing games, 2025. URL <https://arxiv.org/abs/2506.15624>.
- Yupeng Huo, Yaxi Lu, Zhong Zhang, Haotian Chen, and Yankai Lin. Atommem: Learnable dynamic agentic memory with atomic memory operation, 2026. URL <https://arxiv.org/abs/2601.08323>.
- Xun Jiang, Feng Li, Han Zhao, Jiahao Qiu, Jiaying Wang, Jun Shao, Shihao Xu, Shu Zhang, Weiling Chen, Xavier Tang, Yize Chen, Mengyue Wu, Weizhi Ma, Mengdi Wang, and Tianqiao Chen. Long term memory: The foundation of AI self-evolution, 2024. URL <https://arxiv.org/abs/2410.15665>.
- LangChain Team. LangMem. <https://github.com/langchain-ai/langmem>, 2025. Accessed: 2026-07-19.
- Rui Li, Zeyu Zhang, Xiaohe Bo, Zihang Tian, Xu Chen, Quanyu Dai, Zhenhua Dong, and Ruiming Tang. CAM: A constructivist view of agentic memory for LLM-based reading comprehension. In *Advances in Neural Information Processing Systems*, volume 38. Curran Associates, Inc., 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/a488aa1a0c00d76db8a922ef7815a786-Abstract-Conference.html.
- Aixin Liu, Aoxue Mei, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, et al. DeepSeek-V3.2: Pushing the frontier of open large language models, 2025. URL <https://arxiv.org/abs/2512.02556>.
- Xufang Luo, Yuge Zhang, Zhiyuan He, Zilong Wang, Siyun Zhao, Dongsheng Li, Luna K. Qiu, and Yuqing Yang. Agent lightning: Train ANY AI agents with reinforcement learning, 2025. URL <https://arxiv.org/abs/2508.03680>.
- Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. AgentBoard: An analytical evaluation board of multi-turn LLM agents. *Advances in Neural Information Processing Systems*, 37:74325–74362, 2024.
- MiroMind Team. Mirothinker-1.7 & h1: Towards heavy-duty research agents via verification, 2026. URL <https://arxiv.org/abs/2603.15726>.
- Jiangweizhi Peng, Yuanxin Liu, Ruida Zhou, Charles Fleming, Zhaoran Wang, Alfredo Garcia, and Mingyi Hong. HiPER: Hierarchical reinforcement learning with explicit credit assignment for large language model agents, 2026. URL <https://arxiv.org/abs/2602.16165>.
- Hongjin Qian, Zhao Cao, and Zheng Liu. Memobrain: Executive memory as an agentic brain for reasoning. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 2646–2662, San Diego, California, United States, July 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.findings-acl.127. URL <https://aclanthology.org/2026.findings-acl.127/>.
- Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, and Fei Huang. Qwen2.5 technical report. 2024.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Amrith Setlur, Chirag Nagpal, Adam Fisch, Xinyang Geng, Jacob Eisenstein, Rishabh Agarwal, Alekh Agarwal, Jonathan Berant, and Aviral Kumar. Rewarding progress: Scaling automated process verifiers for LLM reasoning. In *International Conference on Learning Representations*, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/hash/98711dea460bdefe0e651ca23ec98ba2-Abstract-Conference.html. Spotlight.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*, 2021. URL <https://openreview.net/forum?id=0IOX0YcCdTn>.

- Ruoyao Wang, Peter Jansen, Marc-Alexandre Côté, and Prithviraj Ammanabrolu. ScienceWorld: Is your agent smarter than a 5th grader? In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing*, pages 11279–11298, Abu Dhabi, United Arab Emirates, December 2022. Association for Computational Linguistics. doi: 10.18653/v1/2022.emnlp-main.775. URL <https://aclanthology.org/2022.emnlp-main.775/>.
- Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, Minhao Cheng, Shuai Wang, Hong Cheng, and Jingren Zhou. Resum: Unlocking long-horizon search intelligence via context summarization, 2025a. URL <https://arxiv.org/abs/2509.13313>.
- Yaxiong Wu, Sheng Liang, Chen Zhang, Yichao Wang, Yongyue Zhang, Huifeng Guo, Ruiming Tang, and Yong Liu. From human memory to AI memory: A survey on memory mechanisms in the era of LLMs, 2025b. URL <https://arxiv.org/abs/2504.15965>.
- Zidi Xiong, Yuping Lin, Wenya Xie, Pengfei He, Zirui Liu, Jiliang Tang, Himabindu Lakkaraju, and Zhen Xiang. How memory management impacts LLM agents: An empirical study of experience-following behavior. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 623–645, San Diego, California, United States, July 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.27. URL <https://aclanthology.org/2026.acl-long.27/>.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*, volume 38. Curran Associates, Inc., 2025. URL https://proceedings.neurips.cc/paper_files/paper/2025/hash/19909c36f51abc4856b4560aff3d36d6-Abstract-Conference.html.
- Sikuan Yan, Xiufeng Yang, Zuchao Huang, Ercong Nie, Zifeng Ding, Zonggen Li, Xiaowen Ma, Jinhe Bi, Kristian Kersting, Jeff Z. Pan, Hinrich Schuetze, Volker Tresp, and Yunpu Ma. Memory-r1: Enhancing large language model agents to manage and utilize memories via reinforcement learning. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12805–12825, San Diego, California, United States, July 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.583. URL <https://aclanthology.org/2026.acl-long.583/>.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chenchen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2369–2380, Brussels, Belgium, October–November 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1259. URL <https://aclanthology.org/D18-1259/>.
- Yi Yu, Liuyi Yao, Yuexiang Xie, Qingquan Tan, Jiaqi Feng, Yaliang Li, and Libing Wu. Agentic memory: Learning unified long-term and short-term memory management for large language model agents. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 21457–21483, San Diego, California, United States, July 2026. Association for Computational Linguistics. doi: 10.18653/v1/2026.acl-long.981. URL <https://aclanthology.org/2026.acl-long.981/>.
- Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19724–19731. AAAI Press, 2024. doi: 10.1609/aaai.v38i17.29946. URL <https://ojs.aaai.org/index.php/AAAI/article/view/29946>.
- Zijian Zhou, Ao Qu, Zhaoxuan Wu, Sunghwan Kim, Alok Prakash, Daniela Rus, Jinhua Zhao, Bryan Kian Hsiang Low, and Paul Pu Liang. MEM1: Learning to synergize memory and reasoning for efficient long-horizon agents, 2025. URL <https://arxiv.org/abs/2506.15841>.

A Method and Tool Specifications

The following sections provide the runtime details omitted from the main paper. We first describe how M_t , C_t , and H_t are maintained during task execution. We then give the tool-specific inputs, preconditions, and state-transition rules of the seven atomic memory tools. The final two subsections explain historical-context recovery and failure handling. Training-instance construction, verifier rubrics, and experimental configurations are provided in the subsequent appendices.

A.1 Runtime State and Execution Protocol

VerMem represents the memory-decision state as $s_t = (q, M_t, C_t, H_t)$. The task specification q remains fixed within one task instance. The long-term memory M_t contains persistent entries that can be reused in later steps or stages. Each entry records its content, source, current status, and revision history. The active context C_t contains the budgeted information currently visible to the task solver. The episodic history H_t preserves the ordered record of observations, task actions, tool outputs, intermediate conclusions, and memory decisions from the same task.

The active context and episodic history serve different purposes. Removing information from C_t reduces the content visible to the task solver, but does not remove the corresponding event from H_t . This separation allows earlier evidence to be recovered after the task focus changes.

At each memory-decision step, the policy generates a complete command $u_t = (v_t, \xi_t)$, where $v_t \in \mathcal{V} \cup \{\emptyset\}$. Each non-null command specifies one atomic tool and its structured arguments. The policy samples u_t from the bounded serialization $x_t = g(s_t)$ defined in Equation (1). The policy receives a bounded serialization of the task specification and budgeted views of M_t , C_t , and H_t . This policy-visible serialization is distinct from the active-context budget governing C_t . Before the command changes the real memory state, the proposed transition is checked against the tool schema, target state, required arguments, source information, and current budget. The transition follows Equation (2). Each valid non-null command commits its realized memory transition. The null command leaves M_t and C_t unchanged. On these states, $\mathcal{T}_m$ is the identity transition. The identity decision is retained in the rollout record and does not create a memory-state modification. If a non-null command is rejected, M_t and C_t remain unchanged, while the failed command and its reason are recorded in H_t . During training, the corresponding violation is recorded through $c_{t,k}$.

After the memory transition, the task solver continues with the resulting active context. The task action and resulting observation are then incorporated through the environment interaction transition in Equation (3), which constructs s_{t+1} . The following procedure summarizes this

runtime process. The local and global verifiers are used only during training and do not participate in the reported inference process.

Unified runtime procedure. Starting from M_0 , C_0 , and H_0 , each memory-decision step performs the following operations:

1. construct $s_t = (q, M_t, C_t, H_t)$ and its bounded serialization $x_t = g(s_t)$;
2. sample the complete command $u_t \sim \pi_\theta(\cdot | x_t)$;
3. compute $\tilde{s}_t = \mathcal{T}_m(s_t, u_t)$ under the applicable execution branch. The valid non-null command commits its realized transition. The valid null command leaves M_t and C_t unchanged, creates no memory-state modification, and retains the identity decision in the rollout record. The rejected non-null command preserves M_t and C_t , records the failed command and reason in H_t , and receives the corresponding $c_{t,k}$ during training;
4. sample the task action b_t from the resulting active context, collect the next observation, and construct $s_{t+1} = \mathcal{T}_e(\tilde{s}_t, b_t, z_{t+1})$;
5. repeat until task termination or a configured rollout limit.

A.2 Atomic Tool Schemas and State Transitions

Each non-null memory command selects one of the seven tools defined in the main paper and supplies its required arguments. Table 6 summarizes the required information, state transition, and principal validity condition of each tool. Optional implementation fields and benchmark-specific budgets are reported with the experimental configuration.

Add. Add stores new information with persistent utility. The new entry records the information and its source. The operation is rejected when the content is empty, unsupported, or duplicates an existing active entry. When an existing entry requires revision, Update is used instead.

Update. Update revises an existing LTM entry. It creates a new version rather than destructively overwriting the previous content. This retains the relation between the earlier and current versions and allows later inspection of how the memory state changed.

Delete. Delete follows soft-deletion semantics. The target entry is excluded from subsequent ordinary retrieval, while its content and deletion record remain available for inspection. The retained record permits later inspection if the entry must be reconsidered.

Tool and Required Information	State Transition	Key Validity Condition
Add: new content and its source	Create a new persistent entry in M_t .	The content is non-empty, supported, and not a duplicate of an active entry.
Update: target entry, revised content, and source	Create a revised version in M_t while retaining the previous version.	The target exists, and the revision is supported by the current evidence.
Delete: target entry and deletion reason	Mark the target entry as deleted without removing its history.	The target exists, and its deletion is justified by the current state.
Retrieve: query	Introduce relevant active entries from M_t into C_t .	Deleted entries are excluded. The operation follows the configured active-context budget.
Filter: context items to remove	Remove the selected low-value items from C_t .	The selected items belong to C_t , and indispensable evidence is preserved.
SelectEpisode: historical fragments to restore	Introduce selected same-task fragments from H_t into C_t .	The fragments belong to the current task history and fit the active-context budget.
Summarize: context items and their summary	Replace selected content in C_t with a shorter summary.	The summary is faithful to its sources and preserves task-relevant information.

Table 6: Compact specifications of the atomic memory tools in VerMem. The table reports the information required by each operation, its state transition, and its principal validity condition.

Retrieve. Retrieve is categorized as an STM tool because it does not change M_t . Instead, it selects active LTM entries and introduces them into C_t . The retrieval query is generated as part of ξ_t . Returned content is committed only when it satisfies the configured active-context budget. Deleted entries cannot be retrieved.

Filter. Filter removes selected low-value content from C_t . Task instructions, the current environment state, and indispensable evidence are protected. Filtering changes the active context but does not remove the original event from H_t .

SelectEpisode. SelectEpisode performs episode-level context selection over the current task history. It identifies earlier observations, tool outputs, or intermediate conclusions that are relevant to the current task state and returns them to C_t . The operation cannot access history from another task instance.

Summarize. Summarize replaces selected content in C_t with a shorter representation. The summary must remain faithful to the source content and preserve information required by the current task. The original events remain available in H_t .

A.3 Historical-Context Recovery via Episode-Level Context Selection

Historical-context recovery concerns information that has left the active context but becomes useful again at a later task stage. It operates over the episodic history of the current task. It is therefore distinct from retrieving persistent information from LTM.

Property	Retrieve	SelectEpisode
Source	Long-term memory M_t	Episodic history H_t
Content	Persistent facts and reusable knowledge	Earlier observations, tool outputs, and intermediate conclusions
Scope	The current LTM state	The current task history
Motivation	Stored knowledge is absent from C_t	Earlier task evidence becomes relevant again
Target	C_t	C_t
Main risk	Irrelevant or outdated persistent information	Redundant or misleading historical fragments

Table 7: Difference between LTM retrieval and historical-context recovery through episode-level context selection.

The system first identifies same-task historical fragments that are relevant to the current task specification and active context. The memory operation policy then selects the fragments to be restored. Selected content is restored to C_t subject to the configured active-context budget and retains its original source and temporal position in H_t .

The two information paths are

$$M_t \xrightarrow{\text{Retrieve}} C_t, \quad H_t \xrightarrow{\text{SelectEpisode}} C_t.$$

Although both operations introduce information into the active context, their information sources and intended uses are different. Table 7 summarizes this distinction.

This separation supports same-task evidence recovery without treating episodic history as another persistent memory store. It keeps Retrieve and SelectEpisode as distinct operations.

A.4 State Validation and Failure Handling

The non-null memory operation is checked before it modifies the real state. The checks cover the selected tool, required arguments, referenced entries, operation preconditions, source support, and available budget. The transition is committed only when these conditions are satisfied.

If a proposed command is invalid, M_t and C_t remain unchanged. The failed command and its reason are retained in the task history. During training, the corresponding structural, safety, or budget violation contributes to $c_{t,k}$. The task solver then continues with the unchanged active context unless the task itself has reached a terminal failure.

Update must preserve the relation between memory versions. Delete must remain soft. Filter must not remove indispensable evidence. SelectEpisode must remain within the current task history. Summarize must reduce context length without changing the supported meaning. Retrieve, SelectEpisode, and Summarize must also satisfy the active-context budget before their transitions are committed.

These execution checks are separate from semantic verification. They determine whether an operation can be executed safely. The local verifier instead evaluates the task relevance, evidence grounding, local progress, and information fidelity of a realized operation. The global verifier supplies only evidence-coherence and terminal-memory-consistency scores after trajectory completion. These scores enter the composite global branch together with programmatic task, supporting-fact-recall, and efficiency components. Both verifiers are disabled during inference.

B Optimization Objective and Loss Scope

This section records the optimization settings and loss scope that supplement the objectives in the main paper. Training-data and rollout construction are detailed in Appendix D; verifier, normalization, and monitoring details are provided in Appendix F.

B.1 Core Training Configuration and Loss Scope

Table 8 summarizes the settings directly related to data construction and trajectory optimization.

During reinforcement learning, all generated tokens belonging to one atomic memory operation share the same $A_{t,k}^{\text{hier}}$. Different memory decisions retain different advantages. The policy objective is applied only to tokens that serialize the complete command $u_{t,k}$, including its structured arguments. Task-solver outputs, environment observations, input-state tokens, and padding tokens do not receive memory-operation credit. Phase A, Phase B, and

Phase C save separate checkpoints. Each phase initializes the next one through policy parameters only.

C Experimental Configuration and Evaluation Protocol

The evaluation details below specify the splits, state initialization, baseline configurations, metric computation, Qwen-Max evaluation, budget-controlled analysis, and multi-seed reporting used in the main experiments. Training-data construction is described in Appendix D, and the verifier and reward implementations are described in Appendix F. All methods are evaluated on the same task instances with the same backbone, active-context budget, and decoding configuration whenever backbone substitution is supported.

C.1 Benchmarks and Evaluation Splits

We evaluate VerMem on ALFWorld, SciWorld, PDDL, BabyAI, and HotpotQA. ALFWorld, SciWorld, PDDL, and BabyAI use their official test splits. HotpotQA uses the official distractor validation split as the held-out evaluation set because it provides both reference answers and annotated supporting facts. The HotpotQA training split is used only for the SFT and reinforcement-learning instances described in Appendix D. Original question identifiers do not overlap between the construction-training pool, the internal development pool, and the held-out evaluation set.

ALFWorld, SciWorld, and BabyAI use environment success as the task outcome. PDDL uses the progress value returned by the benchmark. One HotpotQA question defines one evaluation episode. The question remains fixed as q , while the distractor passages enter the interaction as task observations. Supporting-fact annotations are not exposed to the memory operation policy. They are used for training-instance construction, programmatic supporting-fact recall, verifier inputs, and evaluation. The reference answer is likewise hidden from the memory operation policy.

Table 9 summarizes the evaluation split and reported metric for each benchmark. Every executable instance in the selected split is evaluated. All methods receive the same task order, environment initialization, and information source.

C.2 State Isolation and Episode Initialization

Every task instance uses an isolated memory state. Each episode starts with empty M_0 and H_0 , while C_0 contains only the task instruction and the initial input. The initial input for ALFWorld, SciWorld, PDDL, and BabyAI is the first observation returned by the benchmark. For Hot-

Setting	Value	Setting	Value
SFT train/dev size	32,000 / 4,000	Task batch per update	8
SFT epochs	3	Candidate-group size K	8
SFT learning rate	2×10^{-5}	Phase A/B/C updates	1,000 / 1,000 / 1,500
SFT effective batch	64	Rollout temperature	0.7
SFT warmup ratio	0.03	Rollout top- p	0.95
Maximum gradient norm	1.0	GRPO clip range	0.2
RL learning rate	1×10^{-6}	KL coefficient	0.1
Policy-state serialization limit	8,192	Task-solver response limit	Configured protocol
Memory-command output limit	Configured protocol	Retrieve/SelectEpisode/Summarize	Configured active-context budget
Random seeds	42, 43, 44	Valid null accounting	Horizon: yes; tool calls: no

Table 8: Core training and rollout configuration used by VerMem. Token limits are measured in model tokens.

Benchmark	Evaluation Split	Reported Metric	Evaluation Focus
ALFWorld	Official test split	Success Rate	Long-horizon embodied interaction and environment-state tracking.
SciWorld	Official test split	Success Rate	Scientific procedure execution and maintenance of intermediate observations.
PDDL	Official AgentBoard test split	Progress Rate	Symbolic planning progress and intermediate goal-state maintenance.
BabyAI	Official test split	Success Rate	Instruction following, navigation, and short-term state tracking.
HotpotQA	Official distractor validation split	LLM-as-a-Judge J_{judge}	Multi-hop evidence aggregation, persistent-memory maintenance, and active-context control.

Table 9: Evaluation splits and primary metrics. Supporting-fact annotations are not exposed to the memory operation policy. They are used for training-instance construction, programmatic supporting-fact recall, verifier inputs, and evaluation.

potQA, it contains the question and the passage content currently exposed by the task environment.

The memory states persist only within one episode. After task completion, failure, or budget termination, M_t , C_t , and H_t are released. The vector store, graph state, metadata store, and temporary context of every external baseline are also cleared before the next task instance. Information from one evaluation example can therefore never enter another example.

C.3 Backbones and Inference Configuration

The main experiments use Qwen2.5-7B-Instruct and Qwen3-4B-Instruct. VerMem, VerMem-noVerify, and Base directly use the corresponding backbone. External methods use the same backbone whenever their memory interface supports backbone substitution. The corresponding method descriptions and reported workflows are followed.

Evaluation uses deterministic decoding. The memory operation policy and task solver use temperature 0 and

top- p 1.0. VerMem constructs $x_t = g(s_t)$ under the 8,192-token policy-state serialization limit. The active-context budget applies separately to C_t . The configured protocol determines the task-solver and memory-command output limits. Retrieve, SelectEpisode, and Summarize follow the configured active-context budget. The valid null command does not count as a memory-tool call or in TC, but it remains a memory decision and counts toward the memory-decision horizon.

Returned memory content is committed only when the resulting C_t satisfies the configured active-context budget. Separately, VerMem uses the bounded policy-state serialization in Equation (1); no additional ranking or truncation rule is introduced here. The local and global verifiers are disabled during evaluation.

Table 10 summarizes the shared inference configuration.

C.4 Baseline Configurations

Base uses the same task solver, backbone, response limit, and active-context budget as VerMem, but it has no ex-

Setting	Value	Setting	Value
Policy temperature	0	Policy top- p	1.0
Task-solver temperature	0	Task-solver top- p	1.0
Policy-state serialization limit	8,192	Task-solver response limit	Configured protocol
Memory-command output limit	Configured protocol	Retrieve/SelectEpisode/Summarize	Configured active-context budget
Evaluation seeds	42, 43, 44	Local/global verifiers	Disabled
Memory reset	After every episode	Null-decision accounting	Horizon: yes; TC: no

Table 10: Shared inference configuration. Token limits are measured with the tokenizer of the corresponding backbone.

PLICIT memory operation policy and no external persistent-memory store.

For LangMem, A-Mem, Mem0, Mem0^g, and AgeMem, the corresponding method descriptions and reported workflows are followed. LangMem retains modular storage, search, and consolidation. A-Mem retains dynamic indexing, linking, and memory evolution. Mem0 uses its extraction, consolidation, and retrieval pipeline. Mem0^g uses the graph-based memory workflow described for the method. AgeMem retains its unified LTM/STM tool policy. The final assembled prompt of every method is subject to the shared evaluation context configuration. For VerMem, the policy-state serialization limit and the active-context budget are separate constraints.

VerMem-noVerify and VerMem share the architecture, backbone, complete atomic command space, SFT initialization, Phase A/B/C curriculum, GRPO configuration, and transition constraints. VerMem-noVerify uses the normalized task-outcome advantage and hard constraints. It does not use the local operation advantage or the composite global trajectory advantage. VerMem uses the complete local-global credit assignment.

Table 11 summarizes the role of each compared system. No method receives a reference answer, supporting-fact label, or future observation during inference.

C.5 Metric Computation

Success Rate on ALFWorld, SciWorld, and BabyAI is the number of successful episodes divided by the number of evaluated episodes. PDDL uses the normalized Progress Rate returned by the benchmark and reports it on a 0–100 scale. HotpotQA uses the Qwen-Max LLM-as-a-Judge score J_{judge} , and the main table reports $100J_{\text{judge}}$.

The Average column is the arithmetic mean of ALFWorld SR, SciWorld SR, PDDL PR, BabyAI SR, and HotpotQA $100J_{\text{judge}}$. Every benchmark therefore receives equal weight.

The reward-function ablation additionally reports TN, MQ, and TC. TN is the mean number of online input and output tokens from the memory operation policy and task solver in one HotpotQA episode. MQ evaluates the active entries in terminal M_t against the annotated supporting facts. Soft-deleted entries are excluded. TC is the mean number of non-null memory-tool calls. The null

action $\emptyset$ is not counted in TC, but it remains part of the memory-decision horizon.

C.6 Qwen-Max Evaluation

The HotpotQA J_{judge} and MQ metrics use the fixed **qwen-max-2025-01-25** snapshot. The evaluator is independent of the DeepSeek-V3.2 teacher and the local and global verifiers. It does not generate SFT targets, provide training rewards, or participate in a memory transition.

Qwen-Max follows the configured deterministic evaluation protocol. The configured protocol determines its decoding and output limits. Every example is evaluated independently, and the output must contain one number in $[0, 1]$. Malformed outputs follow the configured retry handling.

The answer judge compares the question, ground-truth answer, and agent answer. Semantically equivalent concise answers can receive full credit. Partially correct answers receive an intermediate score, while irrelevant or contradictory answers receive a score close to zero. The following block gives the complete answer-judge prompt. Bracketed fields in this and subsequent prompt blocks denote values inserted at runtime.

Qwen-Max answer-judge prompt.

You are an independent judge evaluating the correctness of an answer to a question.

Question:
[QUESTION]

Ground-truth answer:
[GROUND_TRUTH_ANSWER]

Agent answer:
[AGENT_ANSWER]

Score the agent answer from 0.0 to 1.0.

1.0: fully correct or semantically equivalent
0.8-0.9: correct with only a minor omission or wording issue
0.6-0.7: partially correct but missing an important element
0.4-0.5: contains some correct information and a major error
0.2-0.3: mostly incorrect with limited relevant content
0.0-0.1: incorrect, contradictory, or irrelevant

Return only one number between 0.0 and 1.0.
Do not provide an explanation.

The MQ evaluator receives the question, reference answer, annotated supporting facts, and active terminal LTM entries. It evaluates supporting-fact coverage, relevance to

Method	Evaluation Configuration
Base	The common task solver without an explicit memory operation policy or external persistent-memory store.
LangMem	Modular memory workflow with storage, search, and consolidation.
A-Mem	Dynamic indexing, linking, and memory-evolution workflow.
Mem0	Extraction, consolidation, and retrieval pipeline.
Mem0 ^g	Graph-based memory workflow described for Mem0 ^g .
AgeMem	Unified LTM/STM tool-based memory policy.
VerMem-noVerify	The matched VerMem control that uses the normalized task-outcome advantage and hard constraints, without the local operation advantage or the composite global trajectory advantage.
VerMem	The complete unified memory policy with local-global verifier-guided credit assignment.

Table 11: Baseline and internal-control configurations. Every method uses isolated task states and the same evaluation backbone whenever backbone substitution is supported.

the question, and irrelevant stored content. The following block gives the complete MQ prompt.

Qwen-Max memory-quality prompt.

You are an independent judge evaluating the quality of long-term memory for question answering.

Question:
[QUESTION]

Reference answer:
[REFERENCE_ANSWER]

Annotated supporting facts:
[SUPPORTING_FACTS]

Active entries in the terminal long-term memory:
[TERMINAL_LTM]

Evaluate the terminal long-term memory on three criteria:

1. Does it cover the supporting facts needed to answer the question?
2. Are the stored entries relevant and factually consistent?
3. Is it free of irrelevant, duplicated, or misleading content?

Score the memory from 0.0 to 1.0.

1.0: complete, relevant, consistent, and free of noise
0.8-0.9: nearly complete with only a minor omission or irrelevant entry
0.6-0.7: useful but missing important supporting content
0.4-0.5: contains some useful evidence and substantial noise
0.2-0.3: limited relevant content with major errors
0.0-0.1: unsupported, irrelevant, or unusable memory

Return only one number between 0.0 and 1.0.
Do not provide an explanation.

MQ is used only for the reward-function analysis and is not included in the Average column of the main results.

C.7 Budget-Controlled Efficiency Evaluation

The efficiency experiment uses Qwen2.5-7B-Instruct and compares A-Mem, Mem0, AgeMem, and VerMem on ALFWorld, SciWorld, and BabyAI. Success Rate is computed separately for the three benchmarks and then macro-averaged with equal weight.

The online token budgets are 1,000, 1,500, 2,000, 2,500, 3,000, 3,500, 4,000, 4,500, and 5,000. Each value is en-

Setting	Value
Backbone	Qwen2.5-7B-Instruct
Benchmarks	ALFWorld, SciWorld, BabyAI
Methods	A-Mem, Mem0, AgeMem, VerMem
Budget range	1,000–5,000
Budget interval	500
S^*	40.0
B^*	2,500
Budget termination	Unfinished episode is unsuccessful
Curve aggregation	Mean and standard deviation over three seeds

Table 12: Fixed settings for the budget-controlled efficiency-performance evaluation.

forced as a hard episode budget. The unfinished episode is counted as unsuccessful when its budget is exhausted.

Online token accounting includes every input and output token from the memory operation policy and task solver. Repeated prompt content is counted again at every call. Online generative memory modules used by an external baseline are also included. Training, offline embedding, index construction, local/global verifier calls, and Qwen-Max evaluation are excluded.

We select $S^* = 40.0$ and $B^* = 2,500$ from the validation curves and fix both values before test evaluation. $SR@B^*$ is the macro-average SR at the 2,500-token budget. $TN@S^*$ is the linearly interpolated online-token budget at which the mean success curve first reaches S^* , using the two adjacent evaluated budget points that bracket the target.

Table 12 summarizes the fixed efficiency protocol. The lines in Figure 2 show the mean across three seeds, and the shaded regions denote one standard deviation.

C.8 Random Seeds and Statistical Reporting

Training and evaluation use seeds 42, 43, and 44. Every seed independently completes SFT, Phase A, Phase B, Phase C, and downstream evaluation. Methods use the

same benchmark instances and task order within one seed.

The main result tables and quantitative appendix tables report the mean across three seeds. The shaded regions in Figures 2 and 4 denote one standard deviation.

Phase C checkpoints are selected only with the internal development set. Official benchmark test results, held-out HotpotQA J_{judge} , and MQ are never used for checkpoint selection. Efficiency thresholds, evaluator prompts, and all decoding settings are fixed before test evaluation.

D Training Data and Stateful Roll-out Construction

The training pipeline below expands the data construction, teacher generation, quality filtering, and stateful trajectory collection procedures introduced in the Training Pipeline. We first describe how the HotpotQA training split is converted into memory-decision states. We then specify the SFT data composition, the frozen teacher prompt, and the filtering protocol. The remaining subsections present phase-specific episode construction and stateful candidate-trajectory collection.

D.1 HotpotQA Training Data and Split

All SFT and reinforcement learning instances are derived from the HotpotQA training split (Yang et al., 2018). Each original example provides a question q , a reference answer, annotated supporting facts, and distractor passages. The question remains fixed within one training episode. The reference answer supports instance construction and terminal task signals, but it is not exposed to the memory operation policy. Supporting-fact annotations are not exposed to the memory operation policy. They are used for training-instance construction, programmatic supporting-fact recall, verifier inputs, and evaluation.

We divide the original question identifiers in the HotpotQA training split into a 90% construction-training pool and a 10% internal-development pool. All states and episodes derived from one original question remain in the same partition. The official HotpotQA validation and test splits are not used to construct SFT data or Phase A, B, or C episodes.

The configured construction protocol converts supporting facts and distractor passages into source-traceable evidence units used to validate the atomic tool schemas. It also constructs the partial, conflicting, duplicated, and obsolete training states needed by Update and Delete. The exact representation fields and transformation rules are determined by that protocol. These constructed states are not treated as native HotpotQA annotations.

The final SFT training set contains 32,000 states, and the internal development set contains 4,000 states. Add, Update, Delete, Retrieve, Filter, SelectEpisode, Summarize, and the null action $\emptyset$ are balanced in both sets. Each

decision contributes 4,000 training examples and 500 development examples. Half of each class is derived from high-precision boundary templates, and half is retained from teacher-generated candidates. At least 25% of each class consists of boundary or negative decision states.

D.2 SFT Data Generation and Teacher Prompt

Boundary templates construct memory-decision states with a well-defined target. They emphasize distinctions that are easy to confuse, including Add versus Update, Retrieve versus SelectEpisode, Filter versus Summarize, and an explicit operation versus $\emptyset$. For example, when a fact is already represented in M_t and no new evidence is available, the target is $\emptyset$ rather than a duplicate Add. When the missing evidence appears only in H_t , the target is SelectEpisode rather than Retrieve.

The frozen DeepSeek-V3.2 teacher expands the diversity of operation choices and structured arguments. For each state, the teacher receives the bounded policy input x_i , the operation-type subspace allowed by the current scenario, and the source evidence already visible in the state. The reference answer and future observations are not provided. Teacher-candidate sampling, decoding, and output limits follow the configured protocol.

The following block gives the complete teacher prompt. The teacher must return exactly one atomic operation with its structured arguments, or $\emptyset$. It must not return an analysis, an explanatory paragraph, or multiple candidate operations.

SFT teacher prompt.

You are a teacher model for constructing supervised examples for VerMem.

Given a task and the current memory-decision state, select exactly one atomic memory operation, or select the null action when no memory change is needed.

Use only information that is already visible in the state. Do not use the reference answer, future observations, or unsupported knowledge.

Available operations:

1. Add
Use only when new information has persistent value and is not already represented in long-term memory.
2. Update
Use when new evidence revises an existing long-term memory entry.
3. Delete
Use only when an existing entry is clearly incorrect, obsolete, duplicated, or unsupported.
4. Retrieve
Use when required information exists in long-term memory but is missing from the active context.
5. Filter
Remove irrelevant active-context items while preserving all evidence needed by the current task.
6. SelectEpisode
Restore task-relevant fragments from the episodic history of the current task.

```

7. Summarize
  Replace overlong active-context content with a shorter
  representation that preserves critical evidence.

8. Null action
  Use when the current memory state requires no change.

Output exactly one of the following forms:

Add(content="...", source_refs=[])

Update(memory_id="...", new_content="...",
        source_refs=[])

Delete(memory_id="...", reason="...")

Retrieve(query="...")

Filter(drop_refs=[])

SelectEpisode(selected_refs=[])

Summarize(source_refs=[], summary="...")

\varnothing

Do not provide analysis, explanations,
markdown, or more than one operation.

Task:
[TASK]

Long-term memory:
[LONG_TERM_MEMORY]

Active context:
[ACTIVE_CONTEXT]

Episodic history:
[EPISODIC_HISTORY]

Allowed operation-type subspace:
[ALLOWED_ACTIONS]

Visible source evidence:
[VISIBLE_EVIDENCE]

```

D.3 SFT Quality Filtering and Optimization

Teacher outputs are not used directly as supervision labels. Every candidate passes seven deterministic checks:

1. the output contains exactly one operation;
2. the operation belongs to the permitted operation-type subspace;
3. all required arguments are present;
4. every referenced memory entry, context item, or historical fragment exists;
5. all references belong to the same HotpotQA training instance;
6. the realized transition satisfies the tool semantics and preconditions in Appendix A;
7. the output contains no answer leakage, future information, or unsupported content.

Any candidate that fails a check is discarded without automatic repair. Valid candidates are then deduplicated

Target Decision	Training	Development
Add	4,000	500
Update	4,000	500
Delete	4,000	500
Retrieve	4,000	500
Filter	4,000	500
SelectEpisode	4,000	500
Summarize	4,000	500
$\emptyset$	4,000	500
Total	32,000	4,000

Table 13: Balanced target-command coverage in the SFT training and internal development sets.

according to their state, target operation, and structured arguments. When two examples define equivalent states and equivalent transitions, only one is retained.

Negative decision states never use an invalid operation as the supervision target. The generator changes information completeness, candidate location, distractor density, or memory consistency so that a superficially plausible operation becomes inappropriate. The target is another legal operation, corrected arguments, or $\emptyset$. A duplicate Add may therefore be corrected to Update or $\emptyset$, while a Retrieve tendency is corrected to SelectEpisode when the missing evidence appears only in H_t .

Table 13 reports the final SFT coverage. Each target contributes 4,000 training examples and 500 development examples.

SFT uses AdamW for three epochs with a learning rate of 2×10^{-5} , a warmup ratio of 0.03, a maximum gradient norm of 1.0, and an effective batch size of 64. The policy-state serialization limit is 8,192 tokens. The configured protocol determines the target-operation output limit. The loss is applied only to the tokens that serialize the complete target command $u_i^* = (v_i^*, \xi_i^*)$. Task descriptions, input states, system instructions, and padding tokens are excluded from the target. Checkpoint selection uses tool accuracy and executable-call rate on the development set. When two checkpoints have the same tool accuracy, the one with the lower invalid-operation rate is selected.

D.4 Phase-Specific Episode Construction

Reinforcement learning follows the parameter curriculum in Equation (4). Each phase initializes its task states independently and transfers only the policy parameters from the previous phase. Table 14 summarizes the data size, operation-type subspace, and episode construction of each phase.

Phase A. Phase A contains 8,000 training episodes and 800 development episodes. The initial M_0 contains the phase-specific persistent state, C_0 contains the current observation, and H_0 begins with the current episode. Add,

Phase	Train/Dev	Operation-Type space	Sub- Episode Construction
Phase A	8,000 / 800	$\mathcal{V}_{\text{LTM}} \cup \{\emptyset\}$	Incremental facts, conflicting evidence, corrections, duplicates, obsolete entries, and a delayed query.
Phase B	8,000 / 800	$\mathcal{V}_{\text{STM}} \cup \{\emptyset\}$	Prepared LTM, distractor-rich active context, same-task episodic history, summarization, and historical-context recovery.
Phase C	12,000 / 1,200	$\mathcal{V} \cup \{\emptyset\}$	Complete tasks with alternating LTM and STM operations; some tasks additionally require historical-context recovery.

Table 14: Phase-specific episode construction. Only policy parameters are transferred between phases; memory states are initialized independently.

Update, Delete, and $\emptyset$ scenario families are sampled uniformly. Supporting facts are revealed incrementally, and each episode ends with a delayed query. The rollout terminates when the delayed query is completed, the token budget is exhausted, or another configured termination condition is reached.

Phase B. Phase B contains 8,000 training episodes and 800 development episodes. Each instance uses supporting documents and distractor documents from the HotpotQA distractor setting. The initial M_0 contains phase-specific entries. The initial C_0 occupies part of the configured active-context budget, and H_0 contains same-task history. Retrieve, Filter, SelectEpisode, Summarize, and $\emptyset$ scenario families are sampled uniformly.

Historical-context recovery cases place required evidence in earlier same-task history while recent episodes contain related distractors. Retrieve accesses M_t , whereas SelectEpisode accesses H_t . Returned content must satisfy the configured active-context budget. Valid null and rejected commands count toward the memory-decision horizon; valid null commands do not count as memory-tool calls.

Phase C. Phase C contains 12,000 training episodes and 1,200 development episodes. Every episode requires at least one LTM operation and one STM operation. Some episodes require historical-context recovery so that SelectEpisode receives supervision in complete tasks. Documents, task feedback, and observations are revealed sequentially. Early Add or Update decisions may affect a later Retrieve, while Filter or Summarize may change the evidence available to subsequent reasoning. Valid null and rejected commands count toward the memory-decision horizon.

All phases use the 8,192-token bounded policy-state serialization defined by Equation (1). The active-context budget applies separately to C_t . The configured protocol determines the task-solver and memory-command output limits.

D.5 Stateful Candidate-Trajectory Collection

At update n of phase $p \in \{A, B, C\}$, the current memory operation policy samples eight tasks. For task q , the gen-

erator constructs the phase-specific initial state $s_{0,p}^{(q)}$ and copies it into $K = 8$ isolated rollouts according to Equation (8). All trajectories in one candidate group share the task, initial M_0 , C_0 , H_0 , and environment condition. They differ only in policy sampling and the resulting operation sequence. Rollouts use temperature 0.7 and top- p 0.95.

At every step, the memory operation policy generates one complete command $u_{t,k}$, including $\emptyset$. The valid non-null command commits its transition through $\mathcal{T}_m$. The null command leaves M_t and C_t unchanged. On these states, $\mathcal{T}_m$ is the identity transition. The identity decision is retained in the rollout record and does not create a memory-state modification. If a non-null command violates a precondition in Appendix A, M_t and C_t remain unchanged, the failure record is added to H_t , and the corresponding $c_{t,k}$ is recorded. The task solver then continues with the resulting active context. Each rollout record retains the bounded policy input $x_{t,k}$, the complete command $u_{t,k}$, the post-memory state $\tilde{s}_t$, the post-interaction state s_{t+1} , and the command-token log probabilities. This preserves the transition distinction made in Equations (2) and (3) without introducing a second transition-record symbol.

Each trajectory terminates when the task solver produces a final answer, the environment terminates, the configured rollout limit is reached, or the token budget is exhausted. Budget-terminated trajectories remain in the candidate group and are processed by the composite global branch and constraint channel. The global verifier contributes only its evidence-coherence and terminal-memory-consistency components.

Stateful collection procedure. For every task in the current update, the collector copies $s_{0,p}^{(q)}$ into K isolated rollouts. Each rollout repeatedly samples $u_{t,k}$ from the current policy conditioned on $x_{t,k}$, realizes or rejects $\mathcal{T}_m$, applies the separate task and environment transition, and stores both $\tilde{s}_t$ and s_{t+1} . The completed trajectories form $\mathcal{G}_{q,p}^{(n)}$ for local and composite-global credit assignment.

Phase A and Phase B each use 1,000 policy updates, while Phase C uses 1,500 updates. With eight tasks and eight trajectories per task, the three phases collect approximately 64,000, 64,000, and 96,000 candidate trajectories, respectively.

E Additional Quantitative Analyses

The analyses below provide detailed comparisons for the main results, component study, verifier ablation, efficiency evaluation, and reward-function ablation. Every value is reported in the main paper or computed directly from the reported results. No additional evaluation setting is introduced.

E.1 Detailed Cross-Backbone Comparison

Table 15 compares VerMem with the strongest external baseline for each benchmark and backbone. With Qwen2.5-7B-Instruct, the absolute gains range from 3.67 points on PDDL to 8.31 points on HotpotQA. With Qwen3-4B-Instruct, the corresponding range is 3.68 to 8.12 points. The average gains remain close across the two backbones, reaching 6.05 and 5.54 points. HotpotQA and SciWorld show the largest improvements in both settings. Both benchmarks involve evidence preservation, intermediate-state maintenance, and active-context control. PDDL shows the smallest gain, but VerMem remains above the strongest external planning baseline under both backbones.

E.2 Effect of Local and Composite-Global Credit Across All Benchmarks

The main credit-branch ablation focuses on ALFWorld, SciWorld, and HotpotQA. Table 16 compares full VerMem with the matched VerMem-noVerify control across all five benchmarks. VerMem-noVerify uses the normalized task-outcome advantage and hard constraints. It does not use the local operation advantage or the composite global trajectory advantage. Adding both credit branches raises four of the five task scores under Qwen2.5-7B-Instruct and raises the Average by 6.25 points. PDDL is the only exception, with a difference of -0.35 points. With Qwen3-4B-Instruct, VerMem improves every task and raises the Average by 6.21 points. The largest gains occur on HotpotQA and SciWorld for both backbones. This pattern is consistent with complementary operation-level and trajectory-level credit.

E.3 Progressive Component Contributions

Table 17 reports the score and marginal gain at every step of the cumulative component study. The transition from Base to +LT produces the largest initial improvement, especially on SciWorld. Enabling the complete LTM/STM tool space yields further gains on all three tasks. Stateful RL with the normalized task-outcome advantage and hard constraints produces a larger gain on HotpotQA than

on the two interactive environments. The final transition from +noV to +V adds the local operation advantage and the composite global trajectory advantage. It improves ALFWorld, SciWorld, and HotpotQA by 6.12, 9.45, and 9.67 points. The progression is consistent with the two credit branches building on unified memory control and stateful training.

E.4 Complementarity of Local and Composite-Global Credit

Table 18 separates the standalone and conditional gains of the two credit branches. The local branch provides the larger standalone gain on ALFWorld and SciWorld, while the composite global branch provides the larger standalone gain on HotpotQA. The conditional differences are consistent with complementary contributions after either branch has been added. Adding composite-global credit after local credit yields further gains of 1.58, 3.58, and 4.49 points. Adding local credit after composite-global credit yields 2.99, 5.43, and 2.57 points. The gains support complementary roles for the two credit branches.

E.5 Pairwise Efficiency Comparison

Table 19 expands the two efficiency metrics into pairwise comparisons. VerMem reduces the token cost required to reach $S^* = 40.0$ by 20.0% relative to AgeMem, 45.5% relative to Mem0, and 51.1% relative to A-Mem. Under the fixed budget $B^* = 2,500$, it improves macro-average SR by 5.30, 8.70, and 10.50 points. The ordering is consistent under both metrics and supports a stronger efficiency-performance frontier for VerMem.

E.6 Reward-Function Effects

Table 20 reports the absolute and relative changes associated with All>Returns. The complete reward raises J_{judge} by 0.097 and MQ by 0.182, which correspond to relative improvements of 18.3% and 37.1%. TN decreases by 168 tokens, or 7.14%, while TC increases by 1.16 calls. The simultaneous increase in TC and reduction in TN is inconsistent with uniform tool-call suppression as the sole explanation. These aggregate metrics do not identify operation-specific contributions.

The reported ablations associate the complete command space, task-outcome stateful RL, and the complete training signal with higher scores across both evaluated backbones. VerMem also attains the stronger reported efficiency frontier. The results support the joint role of unified memory management and multi-granularity credit assignment.

Backbone	Benchmark	VerMem	Strongest External Baseline	Baseline Score	Gain
Qwen2.5-7B-Instruct	ALFWorld	46.30	AgeMem	41.07	+5.23
	SciWorld	43.39	AgeMem	35.55	+7.84
	PDDL	22.06	A-Mem	18.39	+3.67
	BabyAI	65.55	AgeMem	61.42	+4.13
	HotpotQA	62.75	AgeMem	54.44	+8.31
	Average	48.01	AgeMem	41.96	+6.05
Qwen3-4B-Instruct	ALFWorld	53.73	AgeMem	48.97	+4.76
	SciWorld	66.37	AgeMem	59.48	+6.89
	PDDL	38.75	AgeMem	35.07	+3.68
	BabyAI	76.77	AgeMem	72.56	+4.21
	HotpotQA	63.61	AgeMem	55.49	+8.12
	Average	59.85	AgeMem	54.31	+5.54

Table 15: Detailed comparison between VerMem and the strongest external baseline for each benchmark and backbone. Gains are absolute differences on the reporting scale used in the main table.

Backbone	ALFWorld	SciWorld	PDDL	BabyAI	HotpotQA	100 J_{judge}	Average
Qwen2.5-7B-Instruct	+6.12	+9.45	-0.35	+6.34	+9.67		+6.25
Qwen3-4B-Instruct	+5.10	+7.45	+3.99	+4.66	+9.82		+6.21

Table 16: Absolute difference between VerMem and VerMem-noVerify across all tasks. Positive values favor VerMem. VerMem-noVerify uses the normalized task-outcome advantage and hard constraints; it does not use the local operation advantage or the composite global trajectory advantage.

F Verifier, Credit, Reward, and Reproducibility Specifications

The verifier specification below covers the local and global verifiers, operation-wise normalization, the constraint channel, and the command-token loss mask and monitoring protocol introduced in the main paper. The local verifier evaluates one realized atomic memory transition. The global verifier supplies evidence-coherence and terminal-memory-consistency scores after task completion; it does not compute task correctness, supporting-fact recall, or efficiency. Structural validity, state-transition preconditions, safety requirements, and budget limits are handled by the constraint channel rather than the semantic verifier scores. This separation keeps semantic and hard-constraint penalties distinct.

F.1 Verifier Instantiation and Scoring Protocol

The local and global verifiers use frozen DeepSeek-V3.2 instances with separate prompts and input scopes. The local verifier receives the task q , the bounded policy-visible state $x_t = g(s_t)$, the complete command u_t , the realized memory transition, and the source evidence visible at the current decision. It does not receive an unbounded concatenation of H_t , nor does it observe later memory decisions, the final answer, or the global-verifier output. The global verifier is invoked after trajectory termination. Its input contains the task, reference answer, annotated supporting facts, final answer, ordered memory commands,

their realized transitions, the terminal M_t , C_t , and H_t . Online cost is computed programmatically and is not provided as a verifier-produced score.

Verifier-input truncation, decoding, and output limits are determined by the configured protocol. No separate ranking, removal priority, or compression rule is specified here.

Table 21 summarizes the verifier configuration.

Each verifier dimension is scored with an integer in $\{0, 1, 2, 3, 4\}$. Score 0 denotes a contradicted or harmful decision; score 1 denotes weak support with major errors; score 2 denotes a partially justified decision with clear omissions; score 3 denotes a sound decision with minor deficiencies; and score 4 denotes a fully justified decision. Scores are divided by 4 before reward computation.

The implementation performs deterministic parsing and retry handling. The exact fallback after repeated malformed verifier outputs is determined by the execution protocol.

F.2 Local Verifier

The local verifier evaluates one executable complete command and its realized state transition. Executable commands include valid null decisions, which use the separate null rubric below. It scores task relevance, evidence grounding, local progress, and information fidelity. The four scores are divided by 4 and averaged to obtain $r_{t,k}^{\text{local}}$. The trajectory-level local reward in Equation (12) sums the scores produced for executable decisions in $\mathcal{E}_k$ and normalizes them by the full memory-decision horizon T_k . It is used only for monitoring. It does not replace the

Configuration	ALFWorld	Increment	SciWorld	Increment	HotpotQA	100J _{judge}	Increment
Base	27.16	–	13.80	–	38.36		–
+LT	37.73	+10.57	27.98	+14.18	45.76		+7.40
+LT/ST	39.02	+1.29	31.26	+3.28	48.73		+2.97
+noV	40.18	+1.16	33.94	+2.68	53.08		+4.35
+V	46.30	+6.12	43.39	+9.45	62.75		+9.67

Table 17: Numerical decomposition of the cumulative component study. Each increment is measured relative to the preceding configuration. +noV uses the normalized task-outcome advantage and hard constraints. +V adds the local operation advantage and the composite global trajectory advantage and denotes full VerMem.

Comparison	ALFWorld	SciWorld	HotpotQA	100J _{judge}
+Local over VerMem-noVerify	+4.54	+5.87	+5.18	
+Global over VerMem-noVerify	+3.13	+4.02	+7.10	
+Global after +Local	+1.58	+3.58	+4.49	
+Local after +Global	+2.99	+5.43	+2.57	

Table 18: Standalone and conditional gains of the local and composite-global credit branches under Qwen2.5-7B-Instruct. The composite global branch contains programmatic and global-verifier components. +Local adds the local operation advantage. +Global replaces the task-only trajectory advantage with the composite global trajectory advantage.

Method	TN@S*	Reduction	SR@B*	VerMem Gain
A-Mem	4,250	51.1%	33.80	+10.50
Mem0	3,820	45.5%	35.60	+8.70
AgeMem	2,600	20.0%	39.00	+5.30
VerMem	2,080	–	44.30	–

Table 19: Pairwise efficiency comparison under $S^* = 40.0$ and $B^* = 2,500$. Reduction reports the relative decrease in TN@S* achieved by VerMem.

operation-wise normalization in Equation (9) used to compute $A_{t,k}^{\text{local}}$.

Task relevance measures whether the command addresses an actual memory need in the bounded state x_t . Evidence grounding measures whether its content, target, and source are supported by the currently visible evidence. Local progress measures whether the transition improves the current task state. Information fidelity measures whether the operation preserves valid facts, relations, and source attribution.

Table 22 gives the tool-specific conditions for the highest local score. Lower scores reflect increasing departures from these conditions.

Commands rejected by the execution checks in Appendix A are not sent to the local verifier. Their $A_{t,k}^{\text{local}}$ is set to zero. They still receive the trajectory-level A_k^{global} , while the corresponding $c_{t,k}$ supplies the direct decision-level penalty.

The following block gives the local verifier prompt. The operation-specific condition is selected from Table 22.

Local-verifier prompt.

You are the local verifier used during VerMem training.
Evaluate exactly one realized atomic memory transition.

The transition has already passed the structural, reference, precondition, and budget checks.

Use only the task, the state before the command, the selected command, the realized state after the command, and the visible source evidence.

Do not use a later trajectory state, the final answer, or information that is absent from the supplied state.

Score four dimensions with integers from 0 to 4.

0: contradicted, harmful, or entirely unjustified
1: weakly supported with major errors
2: partially justified with clear omissions
3: sound with minor deficiencies
4: fully justified for the current state

RELEVANCE

Does the command address the current memory need?

GROUNDING

Is the command supported by visible source evidence?

LOCAL_PROGRESS

Does the realized transition improve the current task or memory state?

INFORMATION_FIDELITY

Does the transition preserve the valid information required from its sources?

Output exactly four lines and no additional text:

RELEVANCE: <0-4>

GROUNDING: <0-4>

LOCAL_PROGRESS: <0-4>

INFORMATION_FIDELITY: <0-4>

Task q:

[TASK]

Bounded state before the command x_t :

[STATE_BEFORE]

Complete memory command u_t :

[OPERATION]

State after the command:

[STATE_AFTER]

Visible source evidence:

Metric	Answer-Only	All>Returns	Absolute Change	Relative Change
$J_{\text{judge}} \uparrow$	0.531	0.628	+0.097	+18.3%
TN $\downarrow$	2,352	2,184	-168	-7.14%
MQ $\uparrow$	0.491	0.673	+0.182	+37.1%
TC	4.47	5.63	+1.16	+26.0%

Table 20: Absolute and relative effects of the complete reward on HotpotQA using Qwen2.5-7B-Instruct. Negative TN change is favorable.

Setting	Local Verifier	Global Verifier
Model	Frozen DeepSeek-V3.2	Frozen DeepSeek-V3.2
Maximum output	Configured protocol	Configured protocol
Decoding	Configured protocol	Configured protocol
Output dimensions	Relevance, grounding, local progress, and information fidelity	Evidence coherence and terminal-memory consistency
Score range	Integer 0–4	Integer 0–4
Format retry	One deterministic retry	One deterministic retry

Table 21: Configuration of the local and global verifiers. The two verifiers use separate prompts and are disabled during evaluation.

[VISIBLE_EVIDENCE]

Operation-specific condition:
[OPERATION_CONDITION]

F.3 Composite Global Branch and Global Verifier

The composite global branch follows Equations (13) and (14). It combines task correctness, supporting-fact recall, evidence coherence, terminal-memory consistency, and trajectory efficiency. The global verifier produces only evidence coherence and terminal-memory consistency; the other components are programmatic. The four resulting reward terms in Equation (13) are normalized to $[0, 1]$ and averaged with equal weight.

VerMem-noVerify uses the normalized task-outcome advantage and hard constraints. It does not use the local operation advantage or the composite global trajectory advantage.

For HotpotQA, r_k^{task} is the mean of exact match and token-level F1 against the reference answer. Both values are normalized to $[0, 1]$. If no final answer is produced, r_k^{task} is set to zero.

The evidence component uses both annotated supporting facts and global verification. Supporting-fact annotations are not exposed to the memory operation policy. They are used for training-instance construction, programmatic supporting-fact recall, verifier inputs, and evaluation. The exact evidence extraction procedure follows the implementation protocol and is not redefined here. The supporting-fact recall is averaged with the normalized evidence-coherence score returned by the global verifier to obtain r_k^{evid} . Evidence coherence measures whether the final answer follows from a complete, task-relevant, and

non-contradictory evidence chain.

The global verifier also returns terminal-memory consistency. Its normalized score forms r_k^{state} . This dimension checks whether the terminal LTM, active context, and episodic history remain consistent with the observed evidence, whether critical supporting facts remain accessible, and whether obsolete or conflicting entries have been handled. Because H_t is an ordered provenance record, superseded, soft-deleted, rejected, or failed events may remain in history when their status is explicit.

The normalized online cost $\bar{C}_k^{\text{online}}$ uses online tokens, task steps, and memory-tool calls. The three costs are independently min-max normalized within the $K = 8$ candidate trajectories sampled for the same task and then averaged. Each cost component is set to zero when all candidate trajectories have the same value for that component. Teacher calls, verifier calls, Qwen-Max evaluation, and offline index construction are excluded.

The completion threshold is $\delta = 0.5$, as used in Equation (15). The efficiency component is available only when $r_k^{\text{task}} \geq \delta$. Under this gate, a low-cost failed trajectory receives no positive efficiency reward.

The global verifier does not rescore tool syntax, required arguments, or hard preconditions. Task completion, supporting-fact recall, and online cost are computed programmatically. The verifier evaluates only evidence coherence and terminal-memory consistency.

The following block gives the global verifier prompt.

Global-verifier prompt.

You are the global verifier used during VerMem training.

Evaluate the completed task trajectory as a whole.

Answer correctness, supporting-fact recall, online cost, tool syntax, and hard preconditions are computed

Decision	Condition for a Fully Justified Transition
Add	The content has persistent utility, follows visible source evidence, and is not already represented by an active entry in M_t .
Update	New evidence revises an existing entry, the target is correct, and valid information not contradicted by the new evidence is preserved.
Delete	The target is incorrect, unsupported, duplicated, or obsolete, and soft deletion does not remove information that remains necessary for the task.
Retrieve	The returned active LTM entries fill a current information gap in C_t without introducing substantial irrelevant or outdated content.
Filter	The removed items are irrelevant or redundant, while the task instruction, current environment state, and indispensable evidence remain in C_t .
SelectEpisode	The selected fragments belong to the current task history and restore earlier observations, tool outputs, or intermediate conclusions required by the current task state.
Summarize	The operation reduces active-context length while preserving critical facts, relations, and source attribution.
$\emptyset$	The current state requires no memory transition. The null command leaves M_t and C_t unchanged. The identity decision is retained in the rollout record and does not create a memory-state modification.

Table 22: Operation-specific rubric used by the local verifier. Structural validity and hard constraints are evaluated separately.

separately. Do not rescore these items.

Score two dimensions with integers from 0 to 4.

0: contradicted, incoherent, or seriously inconsistent
1: major evidence or terminal-memory failures
2: partially coherent with important omissions
3: coherent with minor deficiencies
4: complete, well supported, and internally consistent

EVIDENCE_COHERENCE

Does the final answer follow from a complete, task-relevant, and non-contradictory evidence chain?

TERMINAL_MEMORY_CONSISTENCY

Are the terminal long-term memory, active context, and episodic history consistent with the observed evidence? Are critical facts preserved and accessible? Are obsolete or conflicting entries handled appropriately? Historical superseded, deleted, rejected, or failed events may remain when their status is explicit.

Output exactly two lines and no additional text:

EVIDENCE_COHERENCE: <0-4>

TERMINAL_MEMORY_CONSISTENCY: <0-4>

Task q:

[TASK]

Reference answer:

[REFERENCE_ANSWER]

Annotated supporting facts:

[SUPPORTING_FACTS]

Final answer:

[FINAL_ANSWER]

Ordered memory commands and realized transitions:

[MEMORY_TRAJECTORY]

Terminal long-term memory:

[TERMINAL_LTM]

Terminal active context:

[TERMINAL_CONTEXT]

Terminal episodic history:

[TERMINAL_HISTORY]

F.4 Advantage Normalization and Hierarchical Credit

The composite global score is normalized within the $K = 8$ trajectories sampled from the same task. Local scores are normalized by operation type $v_{t,k}$, as defined in Equation (9). The seven atomic tools and the null type $v = \emptyset$ maintain separate local statistics.

When an operation type appears at least eight times in the current update, its current-update mean and standard deviation are used. When fewer than eight instances are available, phase-specific running statistics are used. Before each phase, the running statistics are initialized from 128 executable development transitions for every decision type enabled in that phase. The statistics are updated with an exponential moving-average decay of 0.99. They are reinitialized at the start of every phase.

Both local and global normalization use a standard-deviation floor of 0.01, and the ϵ in the main paper is set to 10^{-6} . If every trajectory in a candidate group has the same composite global score, all corresponding global advantages are zero. Local and global advantages are clipped to $[-5, 5]$ before they are combined.

The three coefficients retain the unit values specified with Equation (5). Each rejected command receives $A_{t,k}^{\text{local}} = 0$. Its global advantage remains available because the completed trajectory still determines A_k^{global} . The constraint cost is assigned directly to the corresponding memory decision.

Table 23 summarizes the fixed normalization and optimization settings.

F.5 Auxiliary Constraints

The constraint channel handles discrete and deterministic violations. Table 24 reports the fixed costs and execution behavior. Multiple violations at one memory decision are capped at $c_{t,k} = 1.0$. The valid null decision counts toward the memory-decision horizon used by the auxiliary cost,

Setting	Value	Setting	Value
Candidate-group size K	8	Minimum current-update count	8
Running-statistic initialization	128 per decision	Running-statistic decay	0.99
Standard-deviation floor	0.01	ϵ	10^{-6}
Advantage clipping	$[-5, 5]$	δ	0.5
λ_{local}	1.0	λ_{global}	1.0
$\lambda_{\text{constraint}}$	1.0	ϵ_{clip}	0.2
β_{KL}	0.1	Entropy coefficient	0

Table 23: Fixed normalization, credit-assignment, and stabilization settings used by VerMem.

but it is not a memory-tool call and does not incur a tool-call cost.

Evidence grounding and information fidelity remain continuous semantic scores. The unsupported-operation constraint is triggered only when no valid source exists or the operation directly contradicts visible evidence. The evidence-loss constraint is triggered only when indispensable information is removed and cannot be recovered from M_t or H_t . Partial omissions lower the local score but do not trigger a hard constraint.

F.6 Token-Level GRPO Implementation

The policy ratio is evaluated token by token as $\rho_{t,k,j}(\theta)$ in Equation (10). All generated tokens that serialize one complete command $u_{t,k}$ share the same $A_{t,k}^{\text{hier}}$. The token loss of one command is averaged over its command tokens before losses are averaged over $\mathcal{B}_{\text{op}}$. This gives each command equal weight before batch averaging, independent of argument length.

The loss mask includes only the complete command tokens $y_{t,k,1:L_{t,k}}$, including the operation type and structured arguments. The task specification, M_t , C_t , H_t , system instructions, task-solver outputs, environment observations, and padding tokens are excluded. The null action $\emptyset$ remains a trainable memory decision, and it counts toward the memory-decision horizon. It is not counted as a memory-tool call.

The old policy is frozen before each policy update. The reference policy remains fixed within one phase. Phase A, Phase B, and Phase C use θ_{SFT} , θ_A , and θ_B , respectively, as their reference policies.

The objective uses $\epsilon_{\text{clip}} = 0.2$ and $\beta_{\text{KL}} = 0.1$, as specified in Equation (11). The reinforcement-learning rate is 10^{-6} , the maximum gradient norm is 1.0, and no additional entropy bonus is used. Local and global advantages are clipped to $[-5, 5]$.

F.7 Reward Monitoring and Convergence Analysis

Figure 4 compares the strategy-specific monitoring values of Answer-Only and All>Returns. They use the mappings m_k^{ans} and m_k^{all} in Equations (18) and (17). Since the two

strategies use different monitoring definitions, their absolute values are not required to coincide at the first recorded step.

The complete reinforcement-learning curriculum contains 3,500 policy updates. We record 101 training points. Training Step 0 is the rollout batch collected before the first policy update, and consecutive recorded points are separated by 35 cumulative updates. At every recorded point, the monitoring value is averaged over the 64 candidate trajectories in the current rollout batch. The plotted line is the mean across seeds 42, 43, and 44, and the shaded region denotes one standard deviation across the three runs.

For the common $[0, 1]$ axis, Answer-Only plots m_k^{ans} , while All>Returns plots m_k^{all} , whose global term is r_k^{global} . These mappings are used only for training visualization. They are not policy advantages and do not change the policy objective or checkpoint selection.

The curves use the raw recorded-point statistics without a moving average, temporal smoothing, or spline interpolation. White squares mark every tenth recorded point. The observed curves are consistent with earlier stabilization and lower late-stage variability for the All>Returns configuration, but the monitoring mappings differ and their absolute levels are not directly comparable.

F.8 Protocol-Level Behavior and Failure Analysis

This section organizes protocol-level memory-decision patterns and failure cases implied by the tool semantics and reported ablations. It does not introduce another metric or claim an additional empirical case study.

Phase-Specific Memory Decisions

The three training phases expose different memory needs. Phase A presents incremental facts, corrections, duplicate content, and obsolete entries. The correct decision therefore depends on whether the visible evidence should create, revise, remove, or leave unchanged an entry in M_t . Phase B presents a prepared LTM state, a distractor-rich C_t , and same-task history in H_t . It requires the policy to distinguish Retrieve from SelectEpisode and Filter from Summarize. Phase C combines both tool groups in one

Violation	$c_{t,k}$	Execution Behavior
Invalid tool, missing argument, invalid reference, or failed precondition	1.0	Reject the transition, keep M_t and C_t unchanged, record the failed command and reason in H_t , and continue the task.
Unsupported Add or Update, or an unjustified Delete	1.0	Reject the transition, keep M_t and C_t unchanged, and record the failure in H_t .
Unrecoverable removal of indispensable evidence	1.0	Reject the proposed transition and preserve the previous M_t and C_t ; record the failure in H_t .
Active-context overflow	0.5	Reject the transition, keep M_t and C_t unchanged, and record the failed command and status in H_t .
Memory-decision or interaction-limit violation	1.0	Terminate the trajectory at the configured limit.
Reference-answer or future-information leakage	1.0	Terminate the trajectory and set its task and evidence components to zero.

Table 24: Constraint costs and execution behavior. Costs from multiple violations at one memory decision are capped at 1.0.

trajectory. Early Add or Update decisions may support a later Retrieve, while Filter, Summarize, or SelectEpisode changes the evidence visible to the task solver.

Table 25 summarizes the decision patterns that define these scenarios. The state effects follow the tool semantics in Appendix A.

Historical-Context Recovery

The historical-context recovery scenarios in Phase B make the distinction between M_t and H_t explicit. The initial C_0 is distractor-rich, while H_0 contains same-task history. Retrieve cannot recover evidence that was never written to M_t . SelectEpisode instead identifies the relevant same-task fragment in H_t and restores it to C_t , subject to the configured active-context budget.

Filter may then remove distractors, while Summarize may compress the restored evidence before the task solver continues. This sequence recovers earlier task evidence without treating episodic history as another persistent memory store.

During Phase C, historical-context recovery can occur between LTM maintenance and final reasoning. Early Add or Update decisions may preserve persistent knowledge, while a later SelectEpisode restores an observation or tool output that was useful only after the task focus changed. The combined state transition tests whether the policy coordinates M_t , C_t , and H_t rather than relying on one memory source.

Interpretation of Local and Composite-Global Credit

The two credit branches answer different questions. The local verifier evaluates whether the current executable command u_t is relevant, grounded, useful, and faithful in x_t . The composite global branch combines task correctness, supporting-fact recall, efficiency, and the global verifier’s evidence-coherence and terminal-memory-consistency scores. Table 26 shows how their normalized

advantages treat common trajectory patterns.

The second and third rows explain the main role of hierarchical credit assignment. Trajectory-only supervision cannot separate a useful command followed by a later failure from a poor command hidden inside a successful trajectory. Operation-wise normalization further reduces the risk that tool-specific score ranges dominate the update.

Failure Modes

The failure modes in Table 27 follow directly from the seven tool semantics and the constraint channel. Some errors are semantic and reduce $r_{t,k}^{\text{local}}$. Hard failures reject the transition and contribute to $c_{t,k}$.

The distinction between semantic and hard failures is important. The irrelevant Retrieve can remain structurally valid and therefore receive a low semantic score rather than automatic rejection. Minor but executable omissions in Filter or Summarize are handled the same way. By contrast, unsupported Update, unjustified Delete, protected-evidence removal, and meaning-changing Summarize violate an execution requirement. They are rejected, receive no local semantic evaluation, retain trajectory-level global credit, and receive a direct constraint cost. This separation keeps the semantic and hard-constraint penalties in distinct channels.

Relation to the Quantitative Results

The qualitative patterns are consistent with the experiments in the main paper. Figure 3 shows that LTM operations provide the first major gain, while the complete LTM/STM command space adds further improvement. This progression matches the decisions in Table 25. Persistent storage is useful only when the policy can also retrieve, filter, summarize, or restore information at the correct time.

Table 4 shows complementary gains from the local and composite-global credit branches. The credit patterns in

Memory Need	Decision	Realized State Effect	Targeted Failure Mode
New information has persistent value and is absent from active LTM.	Add	Create a source-bearing entry in M_t .	Loss of information required by a delayed query.
New evidence corrects or refines an active LTM entry.	Update	Create a revised version while preserving the previous version.	Stale or conflicting persistent memory.
The active entry is unsupported, obsolete, or duplicated.	Delete	Soft-delete the target while retaining its history.	Repeated retrieval of invalid or redundant content.
Required information is present in M_t but absent from C_t .	Retrieve	Introduce the relevant active LTM content into C_t .	Reasoning without available persistent evidence.
The active context contains irrelevant or redundant distractors.	Filter	Remove selected low-value items from C_t while preserving their history in H_t .	Context pollution and unnecessary online tokens.
Required same-task evidence exists only in earlier H_t .	SelectEpisode	Restore selected historical fragments to C_t .	Failure of a recent-window or LTM-only recovery strategy.
Relevant context is too long for efficient continued reasoning.	Summarize	Replace selected content in C_t with a shorter faithful representation.	Context overflow or repeated processing of the same evidence.
The current state has no valid memory need.	$\emptyset$	Leave M_t and C_t unchanged and create no memory-state modification. Retain the identity decision in the rollout record. It counts toward the memory-decision horizon, but not toward memory-tool calls or TC.	Unnecessary operations and additional tool cost.

Table 25: Qualitative decision patterns covered by the three-stage training curriculum. Each row links a memory need to the corresponding atomic operation, realized state effect, and targeted failure mode.

Observed Pattern	Local Advantage	Composite-Global Advantage	Credit Interpretation
The command is justified and the completed trajectory is successful.	Positive	Positive	The command and the completed memory strategy are reinforced.
The command is justified, but the trajectory later fails.	Positive	Low or negative	The useful transition retains local credit, while the trajectory-level signal limits its total update.
The command is redundant or weakly grounded, but later steps still produce a correct answer.	Low or negative	Positive	The command does not inherit the full positive credit of the successful trajectory.
The proposed command fails a precondition or hard constraint.	Zero	Trajectory-dependent	The transition is rejected, A_k^{global} is retained, and $c_{t,k}$ is assigned directly to the corresponding decision.

Table 26: Qualitative interpretation of local and composite-global credit. The two advantages distinguish command quality from completed-task utility.

Table 26 illustrate this distinction. Local credit distinguishes the quality of one realized transition. The global verifier evaluates evidence coherence and terminal-memory consistency, while the composite global branch combines those scores with task correctness, supporting-fact recall, and efficiency. The observed gains support complementary roles for operation-level and trajectory-level credit.

The reward ablation in Table 5 reports higher J_{judge} and MQ, lower TN, and more memory-tool calls under All>Returns. This pattern is inconsistent with uniform tool-call suppression as the sole explanation for the lower token count, but it does not identify operation-specific contributions. The stronger efficiency frontier in Figure 2 provides the corresponding system-level result. Detailed numerical decompositions are reported in Appendix E.

Failure	Risk or Observable Consequence	Training Response	Execution Behavior
Duplicate Add	If committed, equivalent information would be stored more than once and could later introduce redundant context.	The rejected command has $A_{t,k}^{\text{local}} = 0$, retains A_k^{global} , and receives $c_{t,k} = 1.0$.	Reject the transition, keep M_t and C_t unchanged, and record the failure in H_t .
Unsupported Update	If committed, the active entry would be revised without visible evidence and could conflict with persistent memory.	The rejected command has $A_{t,k}^{\text{local}} = 0$, retains A_k^{global} , and receives $c_{t,k} = 1.0$.	Reject the transition, preserve the previous version, and record the failure in H_t .
Unjustified Delete	If committed, useful persistent evidence would become unavailable to ordinary retrieval.	The rejected command has $A_{t,k}^{\text{local}} = 0$, retains A_k^{global} , and receives $c_{t,k} = 1.0$.	Reject the transition, preserve the active entry, and record the failure in H_t .
Irrelevant Retrieve	Outdated or task-irrelevant LTM content expands C_t and raises online cost.	Low relevance and local progress; the global efficiency term may also reduce the composite global score. No hard cost is applied when the command remains executable.	Commit only when the operation remains valid and within budget.
Over-aggressive Filter	If protected evidence were removed, the resulting C_t could leave a later reasoning gap.	The hard-rejected command has $A_{t,k}^{\text{local}} = 0$, retains A_k^{global} , and receives $c_{t,k} = 1.0$.	Reject the transition, preserve C_t , and record the failure in H_t when protected evidence is selected.
Lossy Summarize	If committed, the shorter context would alter a supported relation, source, or condition needed by the task.	The hard-rejected command has $A_{t,k}^{\text{local}} = 0$, retains A_k^{global} , and receives the configured hard cost.	Reject the transition, preserve C_t , and record the failure in H_t when the summary changes the supported meaning.
Misleading SelectEpisode	Recent or semantically similar distractors return to C_t instead of the relevant earlier fragment.	Low relevance and local progress; the completed evidence chain may also lower the composite global score. No hard cost is applied when the selected fragments are valid same-task references.	Commit only same-task fragments that fit the active-context budget.
Unnecessary operation	The policy adds tool calls and context changes when $\emptyset$ would preserve a valid state.	Low local progress and potentially lower composite trajectory efficiency; no hard cost is applied when the command is executable.	The transition may remain executable, but receives reduced credit.

Table 27: Representative failure modes and their treatment by local semantic verification, composite-global credit, and hard constraints.